Deep Learning Architectures

Prof. Rama Krishna Sai S Gorthi

Professor,

Department of Electrical Engineering,

IIT – Tirupati, Email: rkg@iittp.ac.in

https://sites.google.com/view/ramakrishnasaigorthi/

भारतीय प्रौद्योगिकी संस्थान तिरुपति

TIRUPATI

Research Areas @ EE Department

Computer Vision–Machine Learning & Medical Imaging (CV-ML-MI)

Power & Control (PC)

Wireless Communications & Networks (WCN)

RF & Microwave (RF)

Electronic Devices & Systems(EDS)

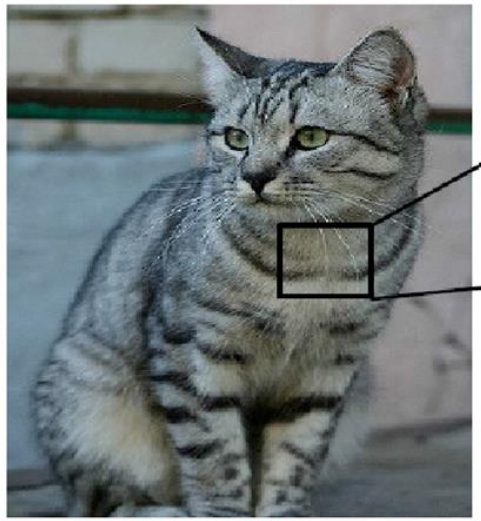
Outline of the talk

- Introduction to Computer Vision and Deep Learning
- Convolutional Neural Networks
- Key Developments and Applications
- Generative Models : GANs and Diffusion Networks
- Discussions & Conclusions

Image Understanding

- Image Processing: Understanding the Computers language

The Problem: Semantic Gap



This image by Nikita is licensed under CC-BY 2.0

```
[[105 112 108 111 104  99 106  90  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  90 115 112 106 103  90  85]
 [ 99  81  81  93 120 131 127 108  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 100  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  78  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  88  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 100 108  92  74  65  72  78]
 [ 89  93  98  97 108 147 131 118 113 114 113 109 106  95  77  88]
 [ 65  77  86  81  77  70 102 123 117 115 117 125 125 138 115  87]
 [ 62  65  82  89  78  71  88 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 128 138 135 105  81  98 118 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 178 157 120  93  88 114 132 112  97  89  55  70  82  99  94]
 [138 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [133 121 103  80  82  88  94 117 145 148 153 102  58  78  93 107]
 [122 164 148 103  71  55  78  83  93 103 119 139 102  61  69  84]]
```

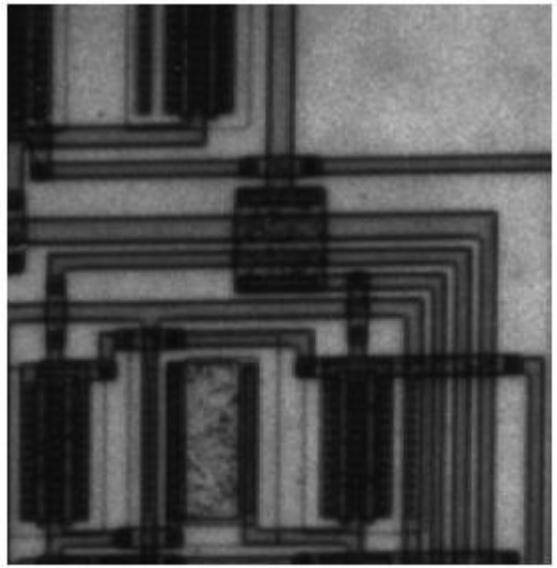
What the computer sees

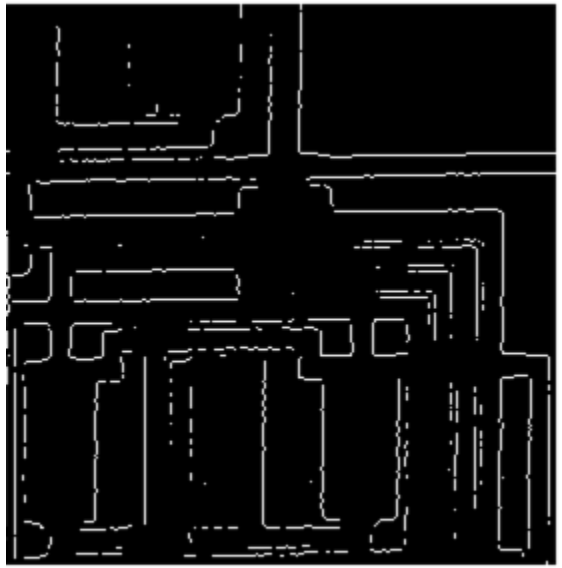
An image is just a big grid of numbers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)

Image Features

- Edge detection By Prewitt filter: $g(x,y) = h(x,y) * f(x,y) + \eta(x,y)$





- Filter for Horizontal edges

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

- Filter for Vertical Edges

Convolution :

$$g(x,y) = f(x,y) * w(x,y)$$

Mask coefficients

w(-1,-1)	w(-1,0)	w(-1,1)
w(0,-1)	w(0,0)	w(0,1)
w(1,-1)	w(1,0)	w(1,1)

Pixels of image

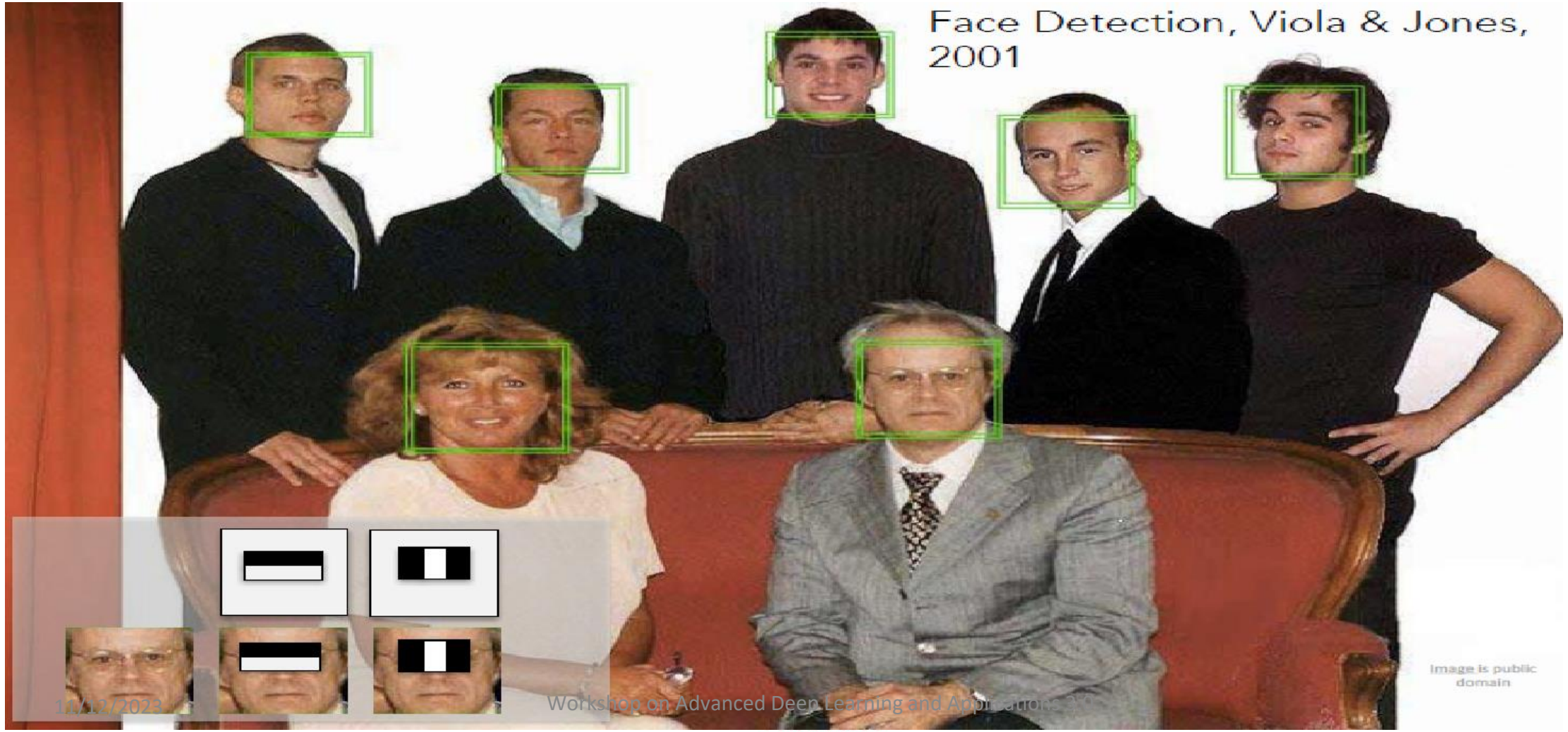
	w(-1,-1) f(x-1,y-1)	w(-1,0) f(x-1,y)	w(-1,1) f(x-1,y+1)	
	w(0,-1) f(x,y-1)	w(0,0) f(x,y)	w(0,1) f(x,y+1)	
	w(1,-1) f(x+1,y-1)	w(1,0) f(x+1,y)	w(1,1) f(x+1,y+1)	

The result is the sum of products of the mask coefficients with the corresponding pixels directly under the mask

$$\begin{aligned} g(x,y) = \; & w(-1,-1)f(x-1,y-1) + w(-1,0)f(x-1,y) + w(-1,1)f(x-1,y+1) + \\ & w(0,-1)f(x,y-1) + w(0,0)f(x,y) + w(0,1)f(x,y+1) + \\ & w(1,-1)f(x+1,y-1) + w(1,0)f(x+1,y) + w(1,1)f(x+1,y+1) \end{aligned}$$

Which information or features to look for ?

Example: Face Detection



Scene Understanding : Object Identification

- Can we automatically count the razors in the given image ?

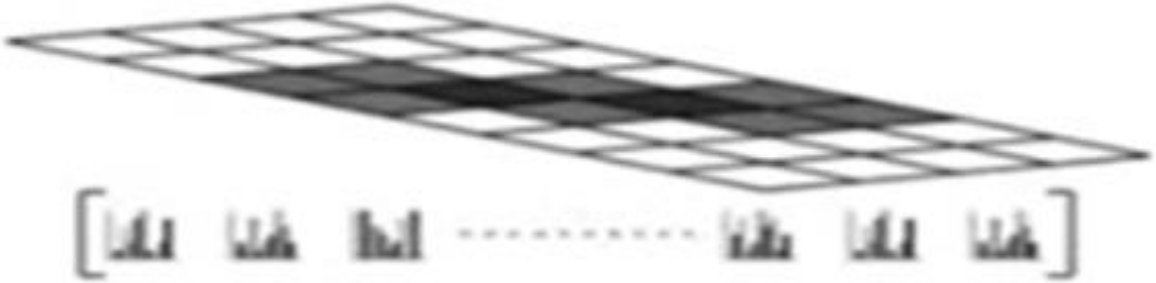
Image is CC0 1.0 public domain

Image : David Lowe, UBC

Way to represent information: Feature Descriptor

- Visualization

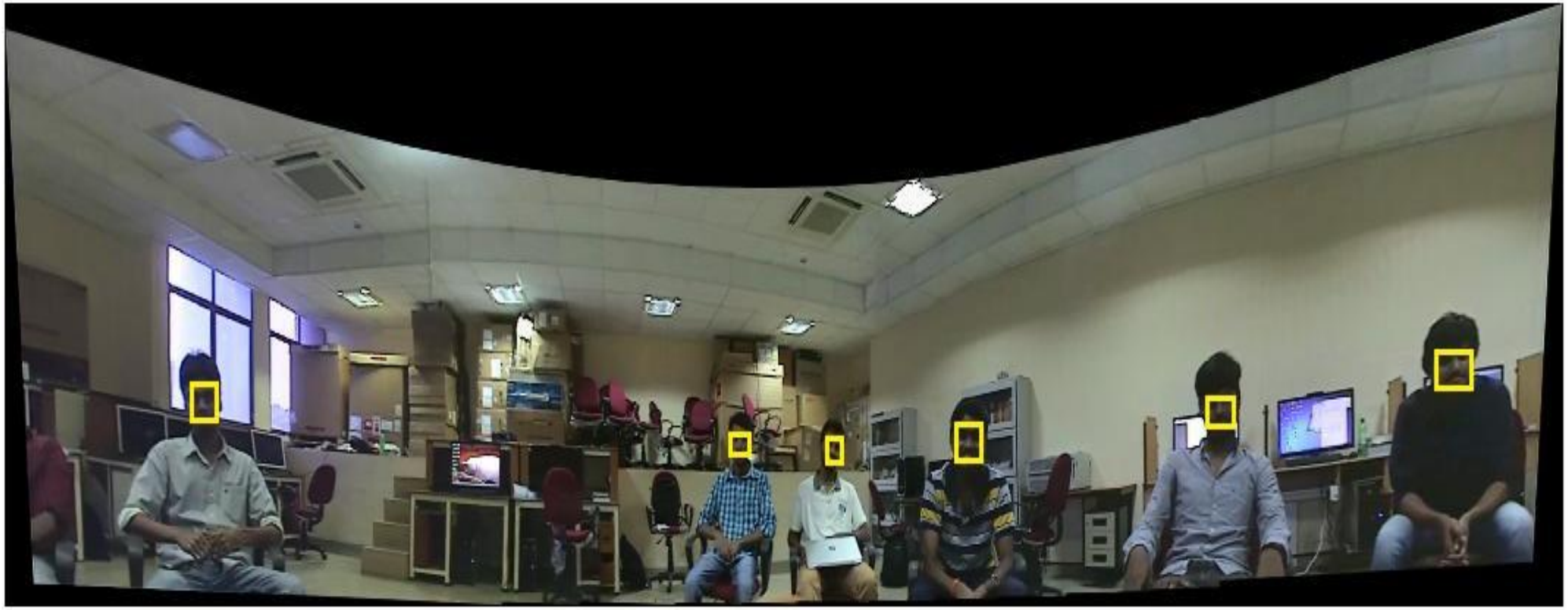
- Concatenate histograms
  - Make it a 1D matrix of length 3780.



N Dalal & B Trigs " Histogram of Orientation Gradients for Human Detection", CVPR 2005.

CV Lab IIST: Stitching and Face Detection

Rahul, B.Tech., IIST

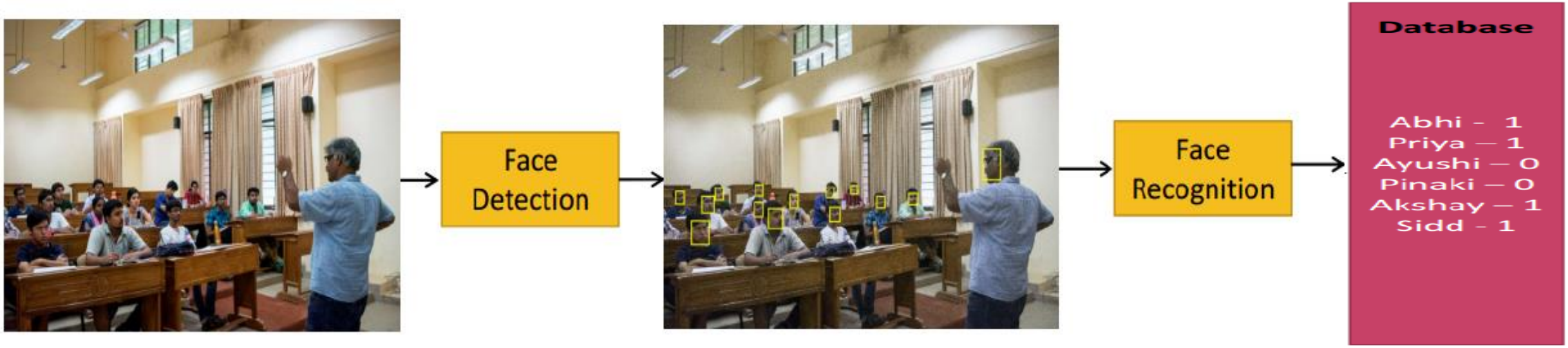


Human-Nonhuman detection

- INRIA data set: http://lear.inrialpes.fr/data
- 1805 images: 64x128
- Few examples:

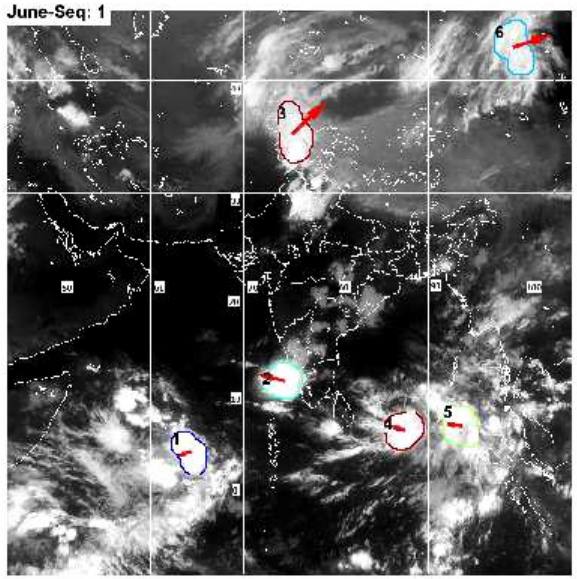
Automated attendance system

Pinaki, B.Tech., IIST



- **Classroom Attendance Project - Akaash Desade & Saketh, IIT Tirupati**

Jay shah, B.Tech; Madan, M.Tech.; Sumitra, B.Tech



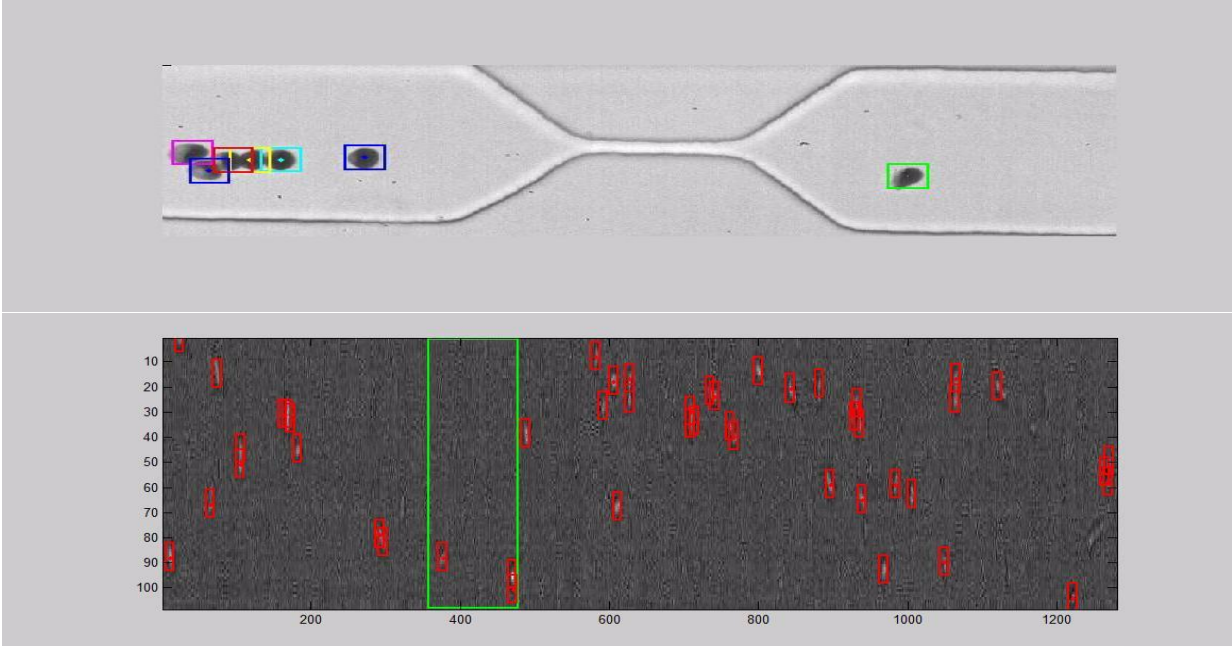
Fast motion and Blur : TLD fails but other two works robustly

Tracking resumption: CT fails but other two works robustly

Non rigid Object Tracking: TLD fails but other two works robustly

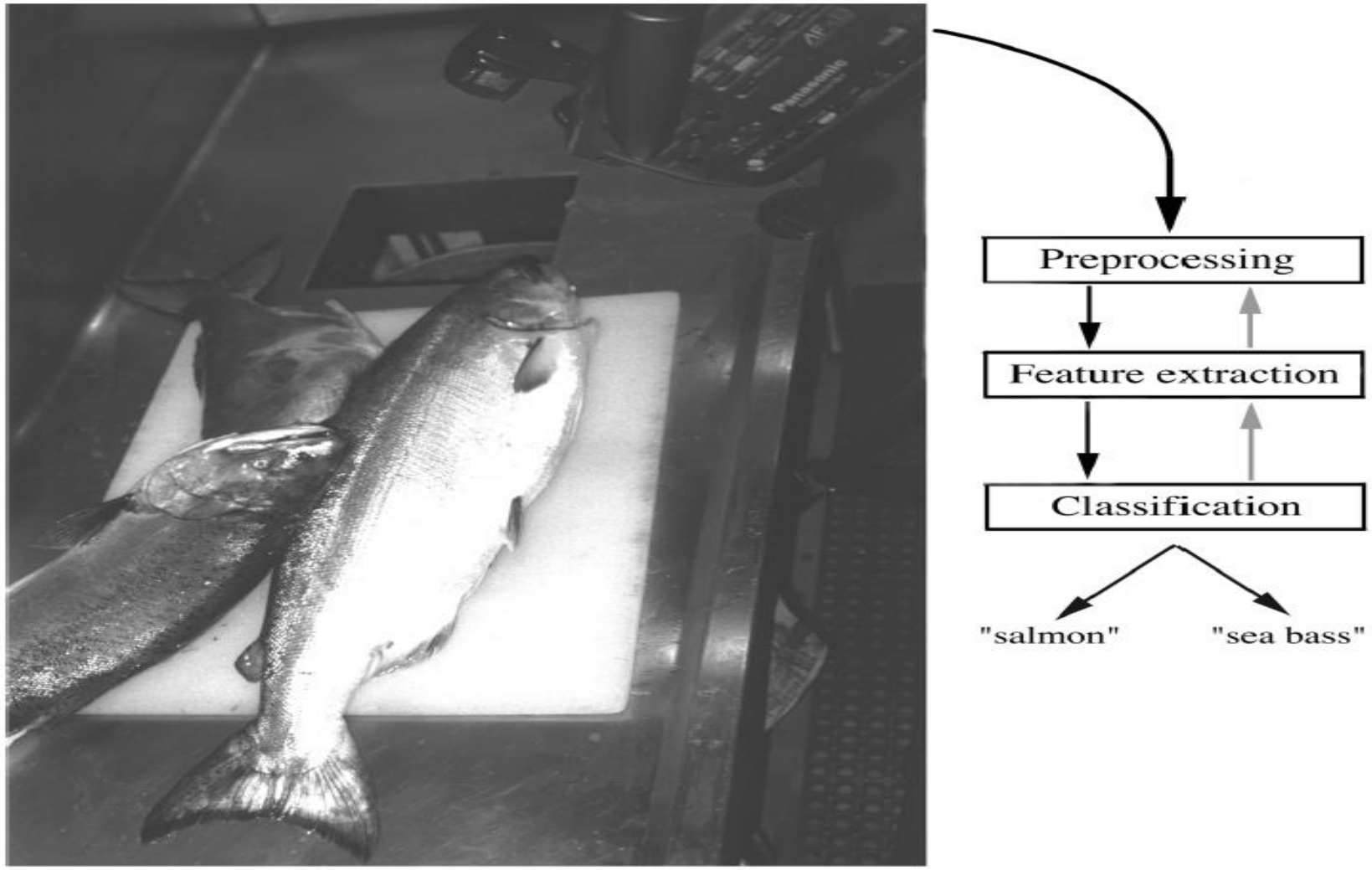
Full out of plane rotation : TLD fails but other two works robustly

Counting RBC& bacteria cells :

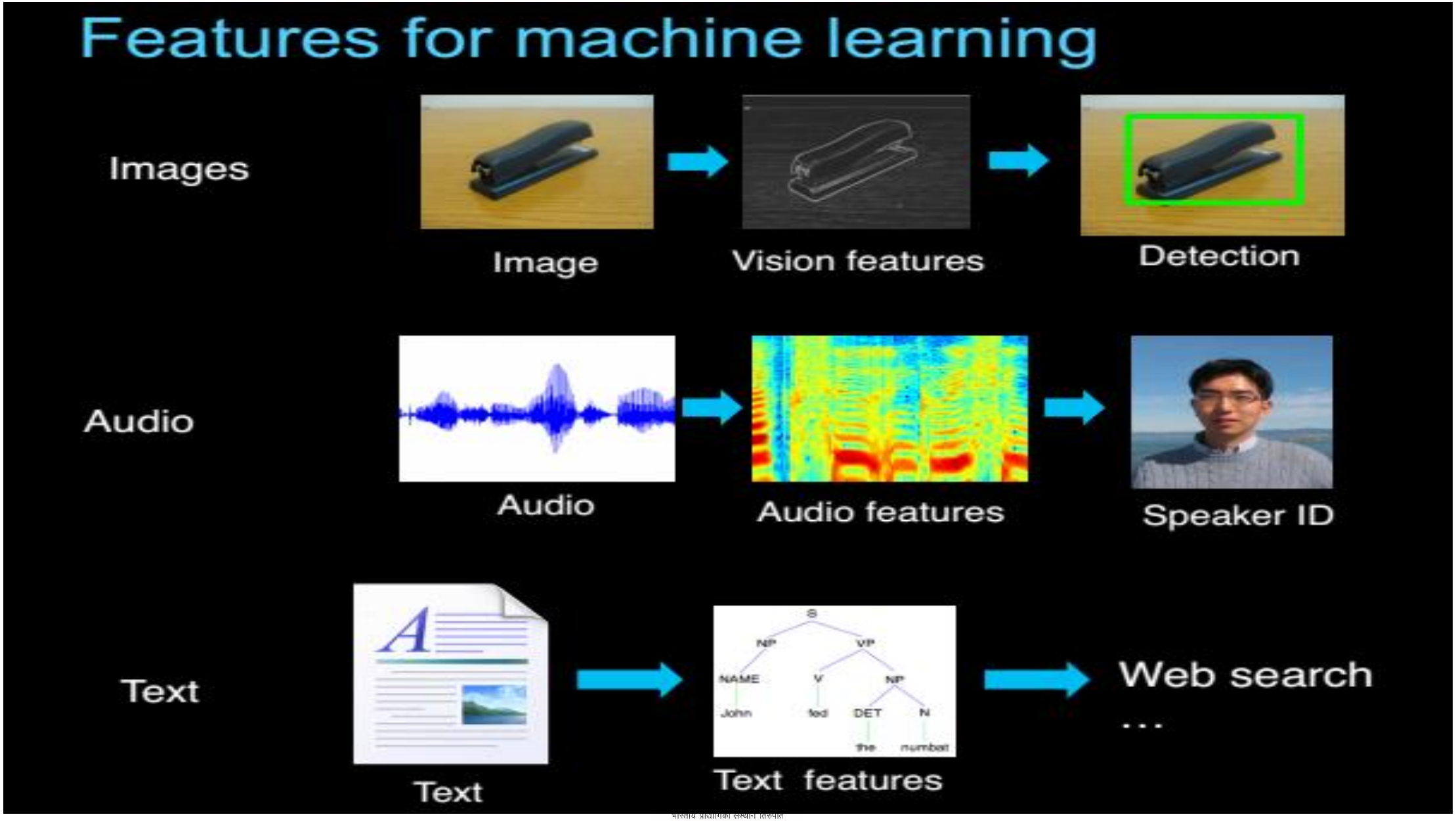


Object follower :

An example task with Machine Learning

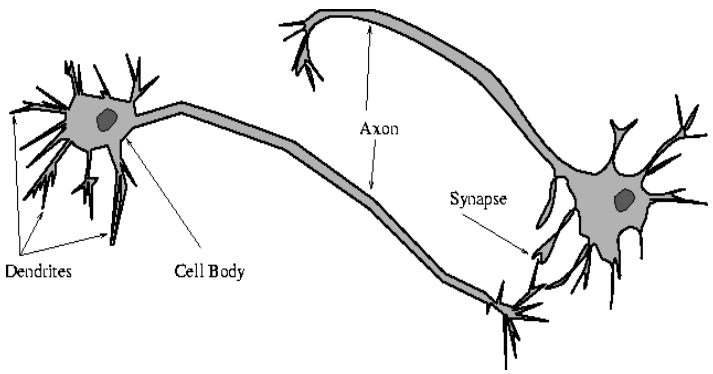


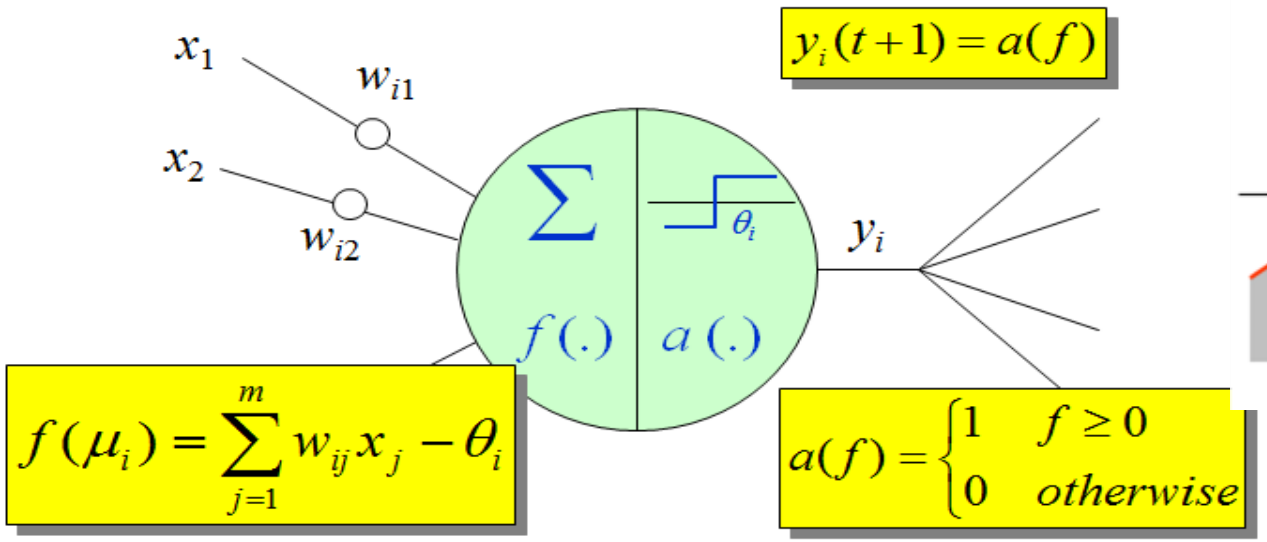
Features for machine learning

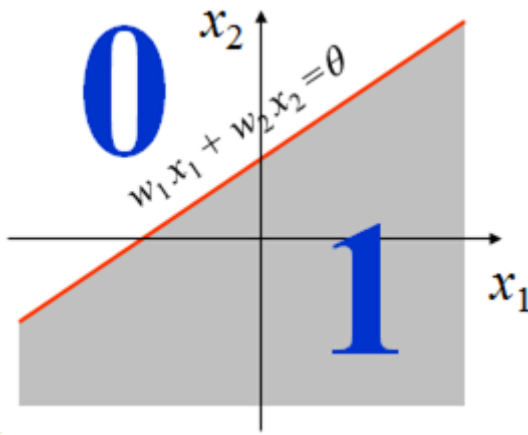


Biological Vs Artificial Neuron

Neuron	Its model (Artificial neuron)
Strength of synapse	weights
Cell body	Summation and transfer function
Signal on axon of neuron	output





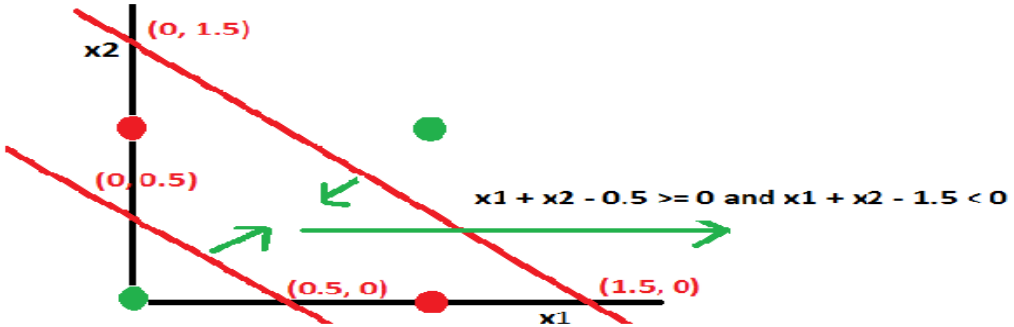


Can X-oR Problem be solved by Perceptron?

- Perceptron implements linear equation in the feature space.
- In 2D perceptron solution represent a line.
- No line can separate different classes of XOR problem.
- However, multi layer neural network can perform the task.

XOR

X1	X2	O
0	0	0
0	1	1
1	0	1
1	1	0



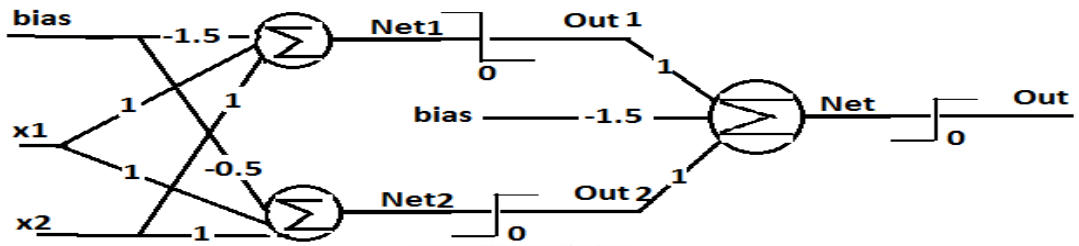
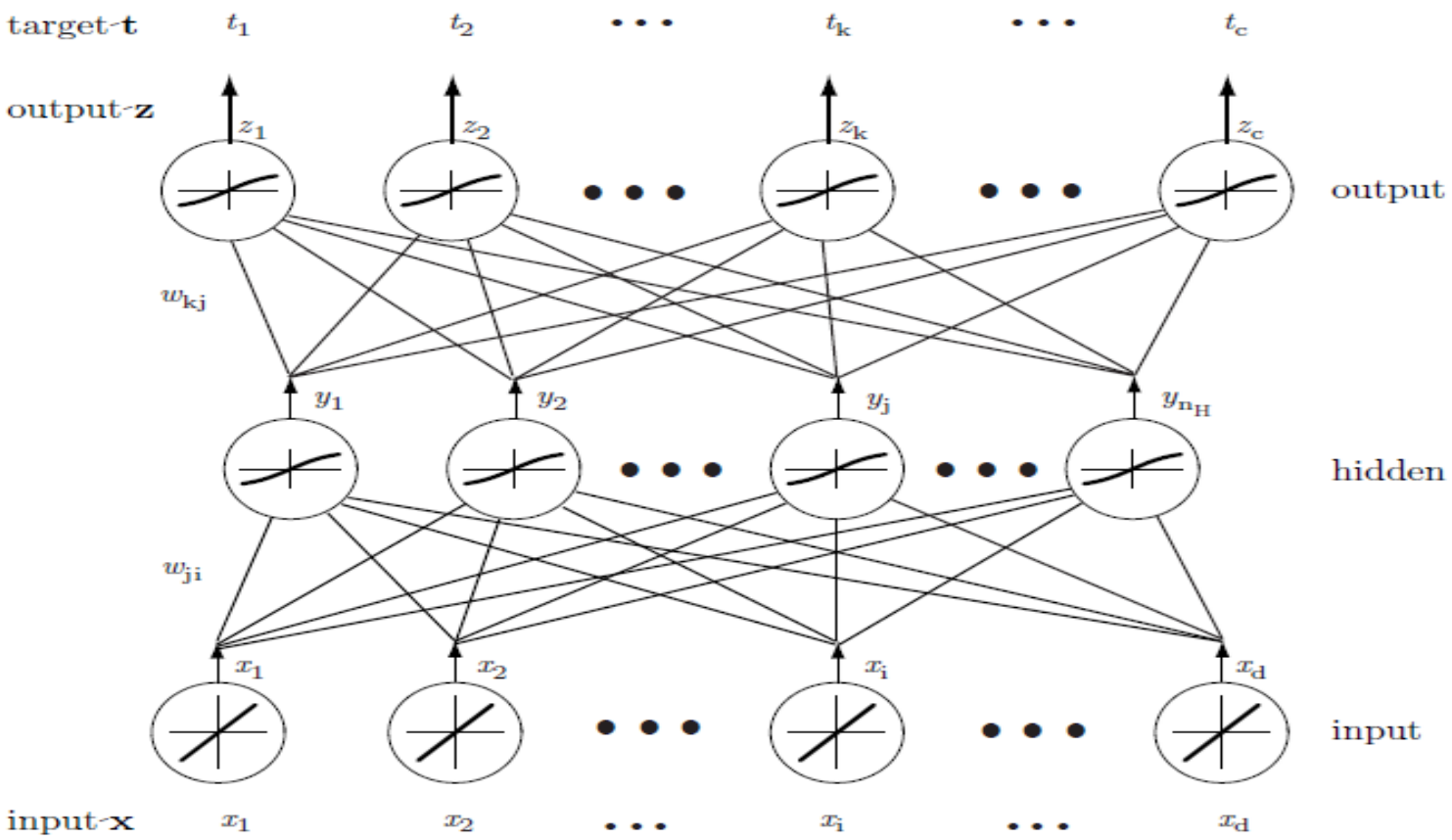


Fig Ref: Pattern Classification, R. O. Duda & P. E. Hart, D. G. Stark 2001.

Feed Forward Neural Network



$$J(\mathbf{w}) \equiv 1/2 \sum_{k=1}^{c} (t_k - z_k)^2$$

$$z_k = f(net_k)$$

$$\frac{\partial J}{\partial w_{kj}} = \frac{\partial J}{\partial net_k} \frac{\partial net_k}{\partial w_{kj}} :$$

$$\Delta w_{kj} = \eta \delta_k y_j = \eta (t_k - z_k) f'(net_k) y_j .$$

Fig Ref: Pattern Classification, R. O. Duda & P. E. Hart, D. G. Stark 2001.

Error back propagation & weight update in deep feed forward NN:

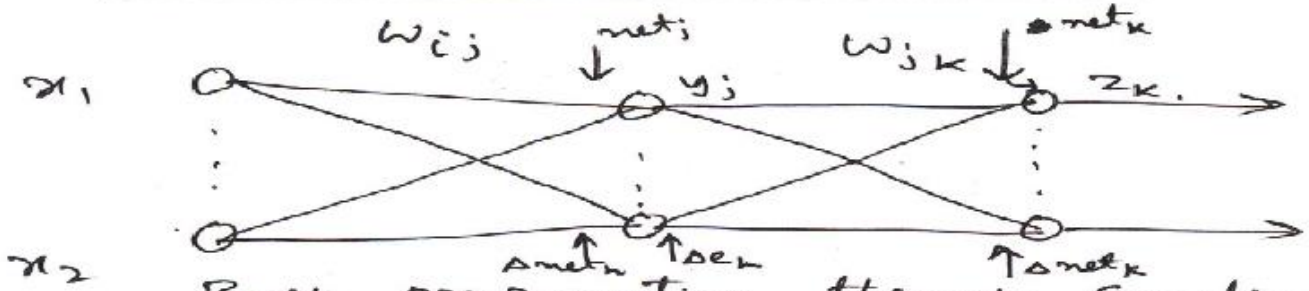
$$J = \frac{1}{2} \sum_k (z_k + d_k)^2$$

$$z_k = f(net_k)$$

$$net_k = \sum_j w_{jk} y_j + b$$

$$y_j = f(net_j)$$

$$net_j = \sum_i w_{ij} x_i + b_j$$



Back propagation through Gradiant descent:

$$w_{jk}(t+1) = w_{jk}(t) - \eta \frac{\partial J}{\partial w_{jk}}$$

$$\Delta w_{jk} = \frac{\partial J}{\partial w_{jk}} = \frac{\partial J}{\partial net_k} \cdot \frac{\partial net_k}{\partial w_{jk}}$$

$$= -(z_k + d_k) f'(net_k) \; y_j$$

$$= -e_k f'(net_k) \; w_{jk} = -\Delta net_k \cdot y_j$$

error matrix

$$\Delta w = -\Delta net_o \cdot y$$

(i) error back propagation (Say column vector)

(ii) to be multiplied with input to get weight updation. (taken row vector)

$$\Delta net_o = \begin{bmatrix} \Delta net_1 \\ \Delta net_2 \\ \vdots \\ \Delta net_n \end{bmatrix} \qquad y = [y_1 \; \cdots \; y_n]$$

$$\Delta w_{kj} = \Delta net_k \cdot y_j$$

Now let us see this for the next layer weights in FFN: w_{ij}

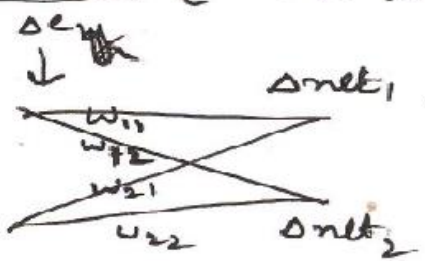
we have $\Delta w_{ij} = \frac{\partial J}{\partial w_{ij}} = \frac{\partial J}{\partial y_j} \cdot \frac{\partial y_j}{\partial w_{ij}}$

$$\Delta w_{ij} = - \sum_k (d_k - z_k) \, f'(net_k) \cdot w_{jk} \cdot f'(net_j) \, x_i$$

$$\Delta w_{ij} = - \sum_k w_{jk} \, \Delta net_k \cdot f'(net_j) \, x_i$$

$$\begin{bmatrix} net_1 \\ net_2 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$$

$$\Delta w^h = \underbrace{w^o \, \Delta net_o \, f'(net^h)}_{\Delta net^h} \cdot x$$



$$\Rightarrow \begin{pmatrix} \Delta e_{w_1} \\ \Delta e_{w_2} \end{pmatrix} = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix} \begin{bmatrix} \Delta net_1 \\ \Delta net_2 \end{bmatrix} \leftarrow \begin{cases} \Delta e_{w_1} = \Delta net_1 \, w_{11} + \Delta net_2 \, w_{12} \\ \Delta e_{w_2} = \Delta net_1 \, w_{21} + \Delta net_2 \, w_{22} \end{cases}$$

Automated Gradients

→ error at prefinal layer.

→ back prop through sigmoid function. (Activation)

$$\Rightarrow \quad \Delta w^h = \underbrace{w^o \, \Delta net_o \, \cdot \, f'(net^h)}_{\text{element by element product}} \, . \qquad x \rightarrow \text{input.}$$

column vector

↑ row vector of inputs $[x_1 \;\; x_2 \; \cdots \; x_d]$.

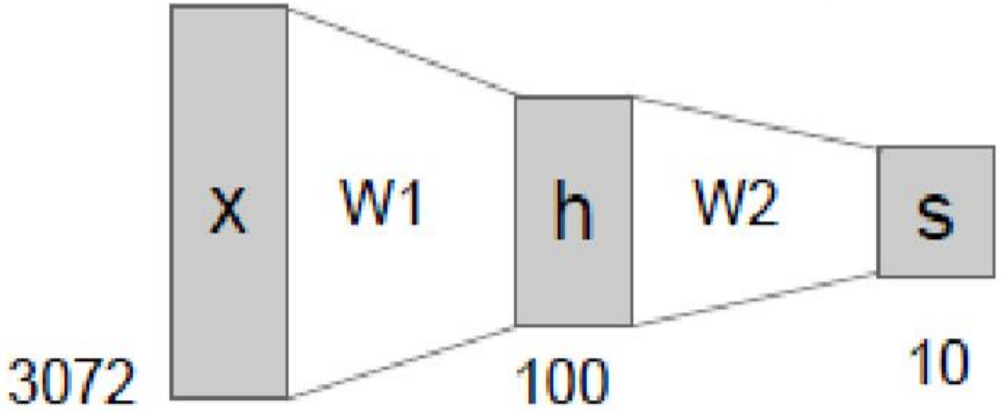
$$= \Delta net_h \cdot x.$$

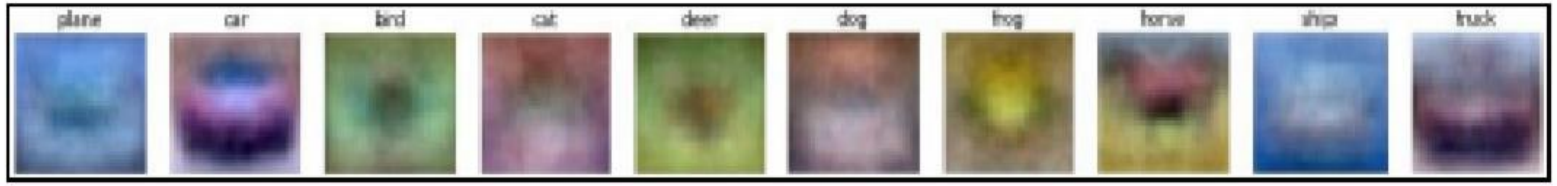
$$\hookrightarrow \begin{bmatrix} \Delta net_{h_1} \\ \Delta net_{h_2} \\ \vdots \\ \vdots \end{bmatrix}.$$

Vector-Matrix formulation

Linear score function: $f = Wx$

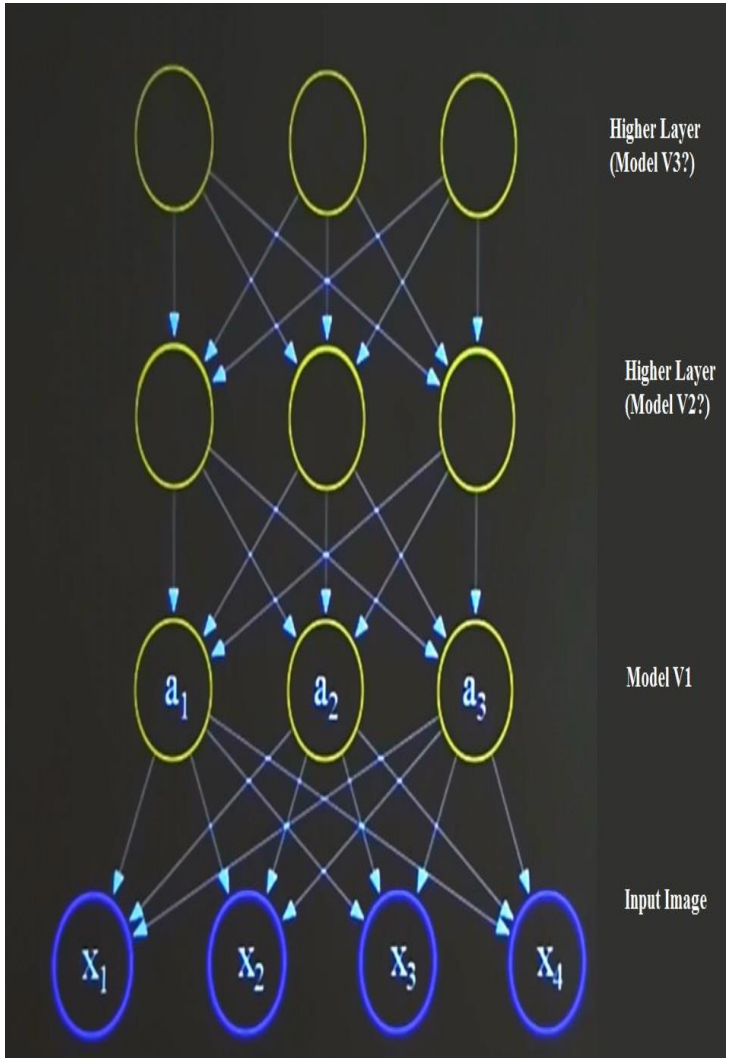
2-layer Neural Network $f = W_2 \max(0, W_1 x)$





Slide Credit: Prof. Fei Fei Li, Stanford

Limitations: Deep NNs



Leraning in Deep Neural Nets:
Forward Pass:

$$y = f_4\left(W_4\, f_3\left(W_3\, f_2\left(W_2\, f_1(W_1, x)\right)\right)\right)$$

Back Propagation:

$$\Delta W_4 = f_4' \;.*\; f_3' \;.*\; f_2' .*\; f_1' .* \cdots$$

Case study: Sigmoid Activation

$$0 < f(x) < 1$$
$$0 < f'(x) = f(x)(1 - f(x)) < 0.25$$
$$\Delta W_4 = f_4' \;.*\; f_3' \;.*\; f_2' .*\; f_1' .* \cdots \approx 0$$

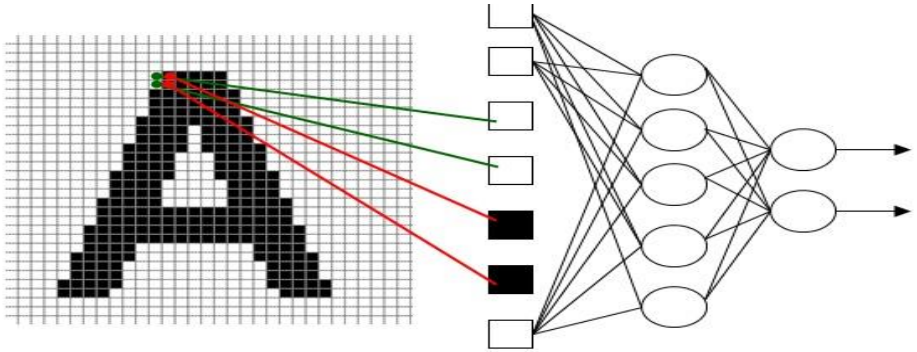
ReLU Activation

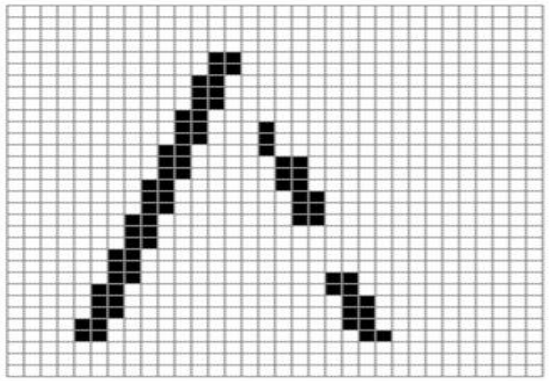
$$y = \max(\mathrm{W}_4 \max(W_3 \max(W_2 \max(W_1, 0), 0), 0), 0)$$

Slide Credit: Prof. Andrew Ng, Stanford

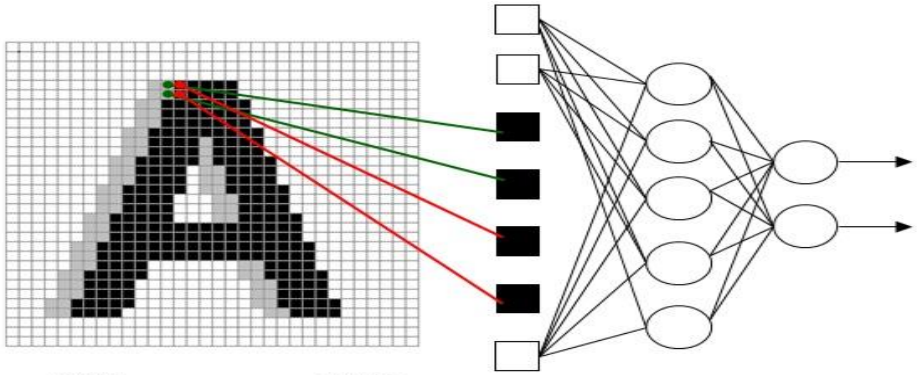
Limitations of ANN

- Computationally expensive for large dimensional features
- Single hidden layer NN cannot deal with shift and distortions even if the problem is linearly separable
  - Need to train with all possible cases?





154 input change
from 2 shift left
77 : black to white
77 : white to black

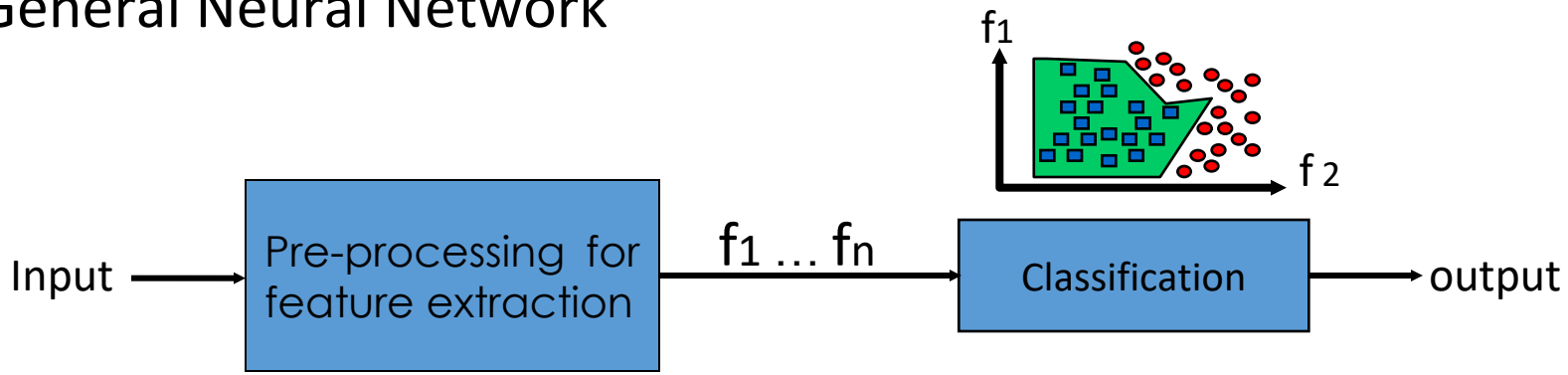


scaling, and other forms of distortion

Fig Ref: Andrew Ng

Classification

- General Neural Network



- Convolutional Neural Networks

Low level features to High level or abstract features

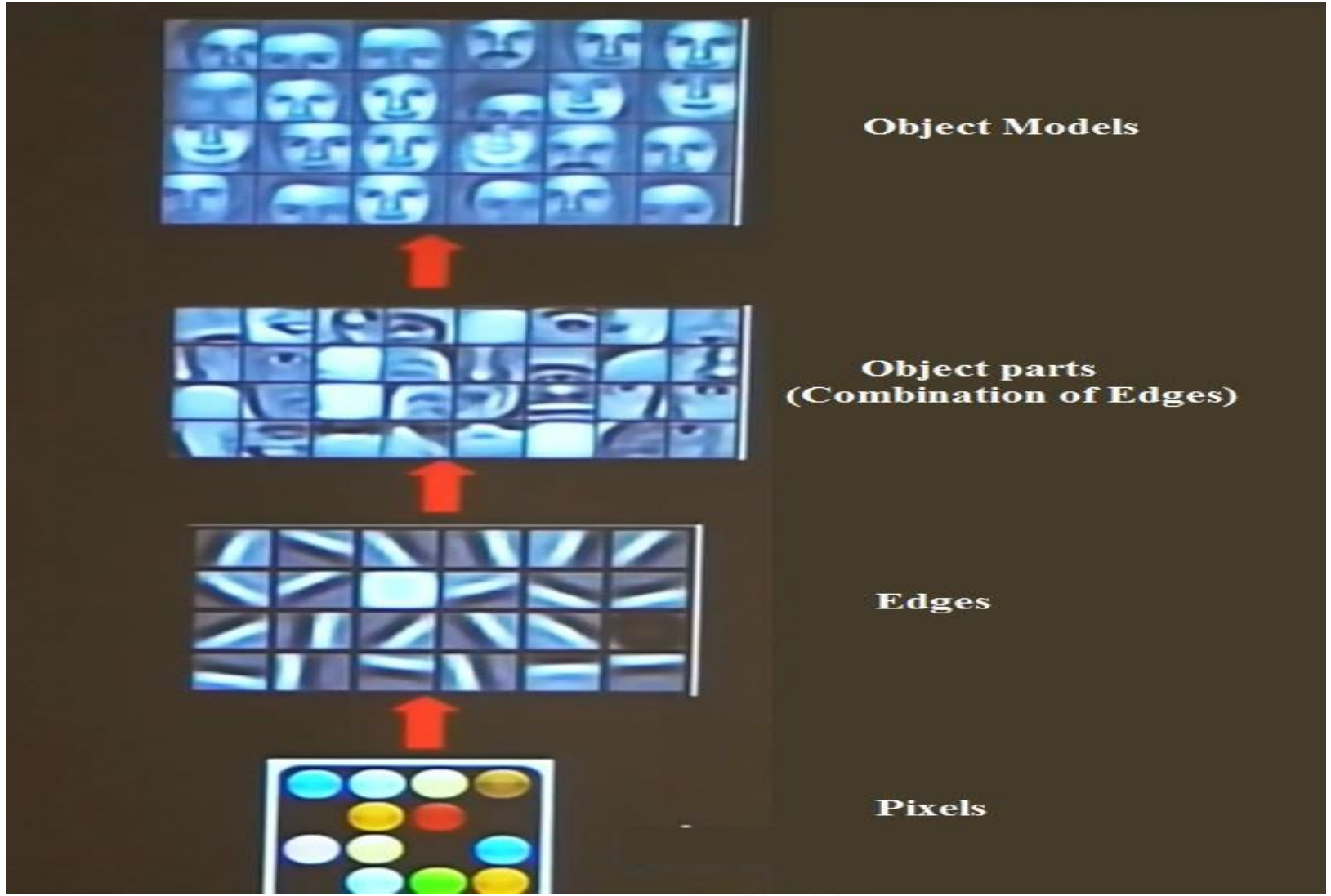


Fig Ref: Andrew Ng

Convolution for feature extraction

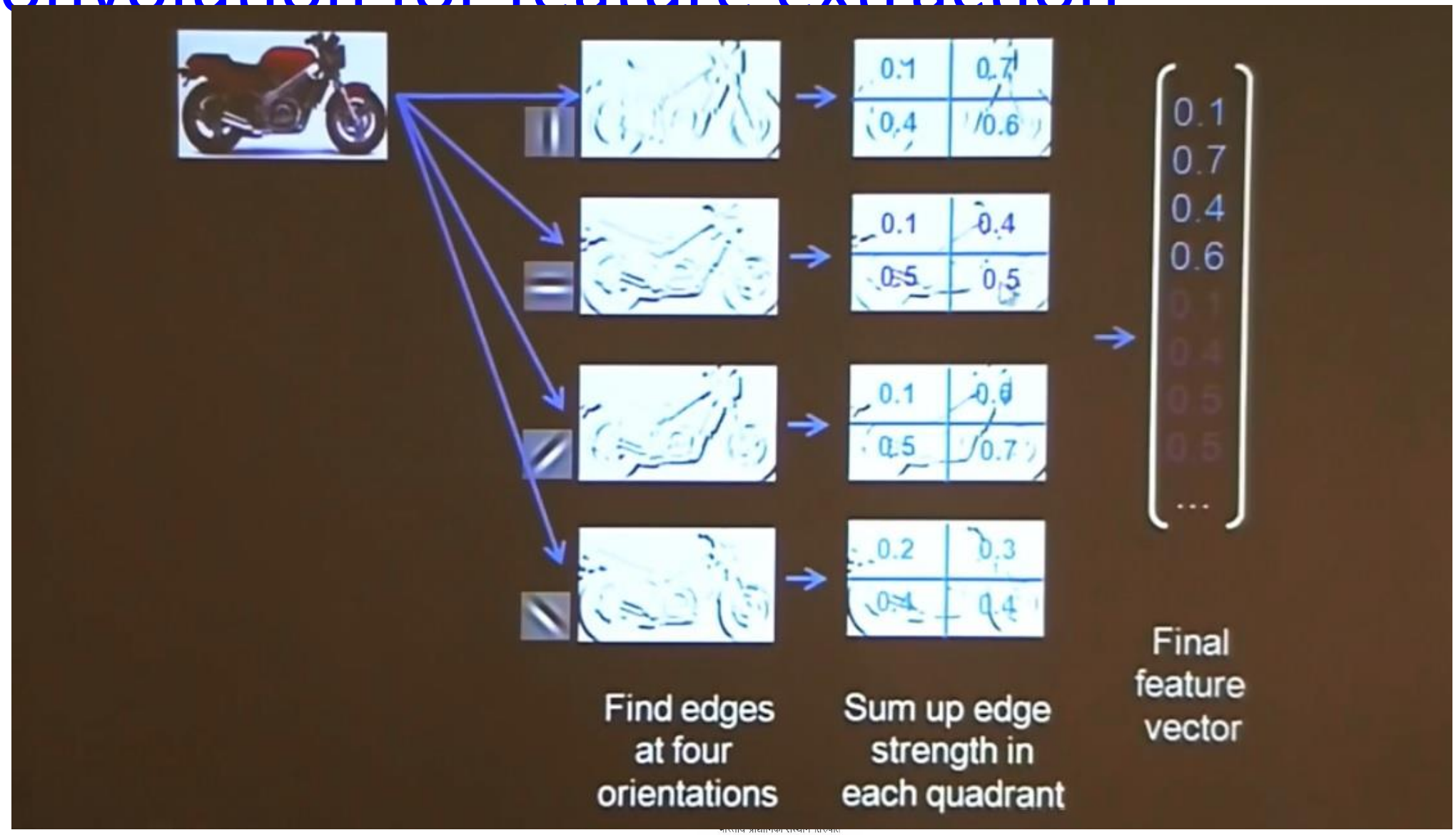
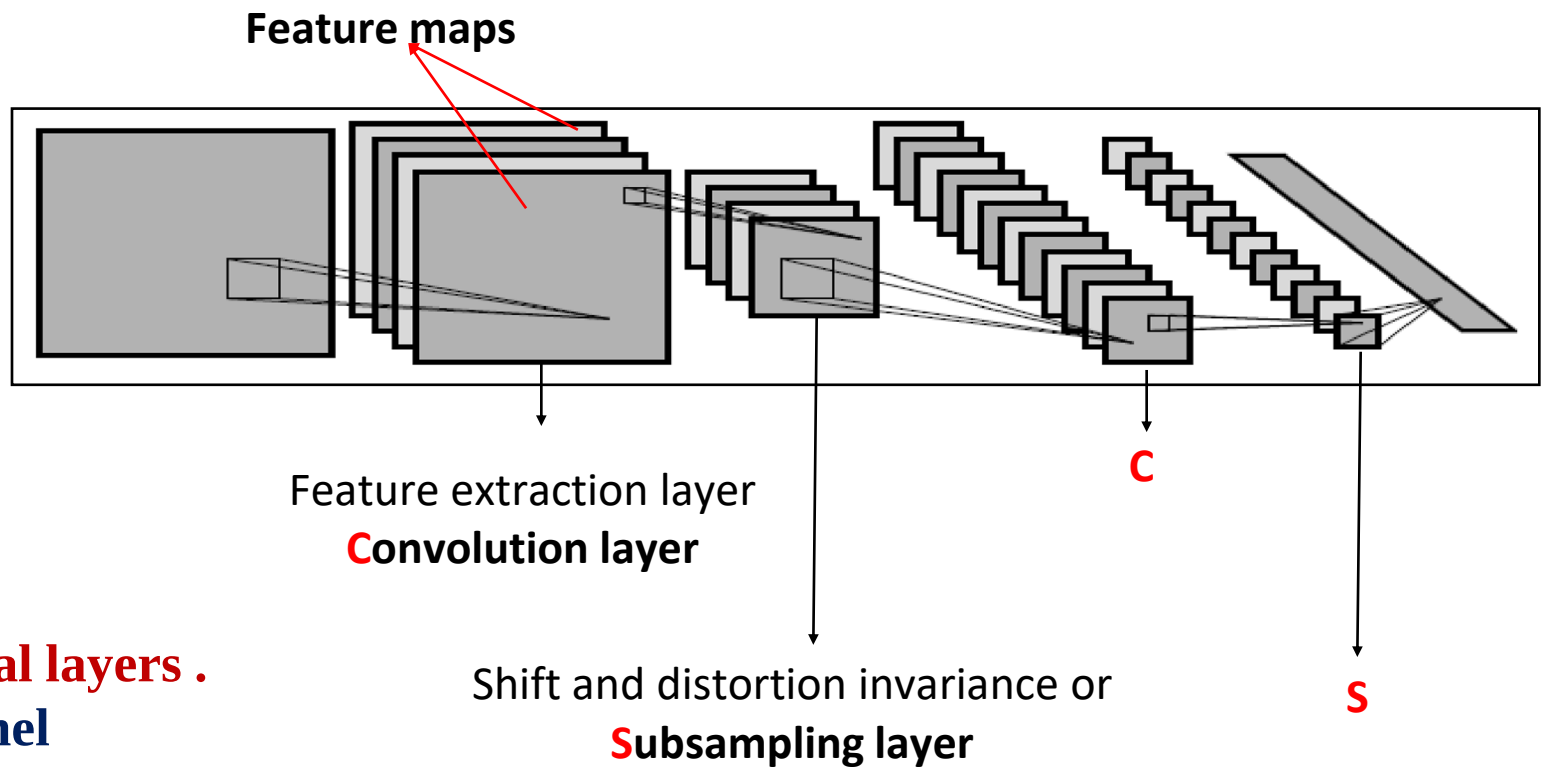


Fig Ref: Andrew Ng

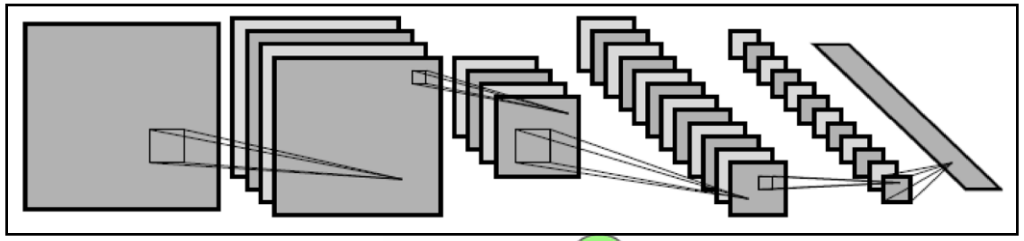
CNN'S Topology

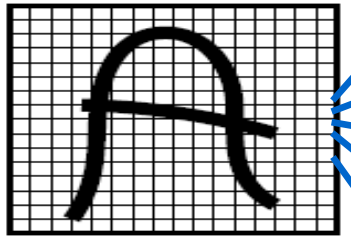


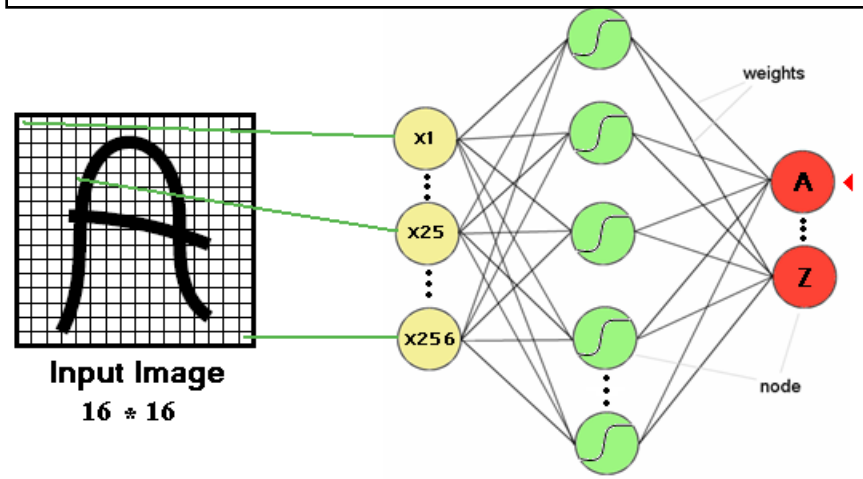
- **Convolutional layers .**
  - **-Kernel**
  - **-Feature Map or filter .**
  - **-Shared weights .**
- **Subsampling or pooling .**
- **Full connected layer(classification)**

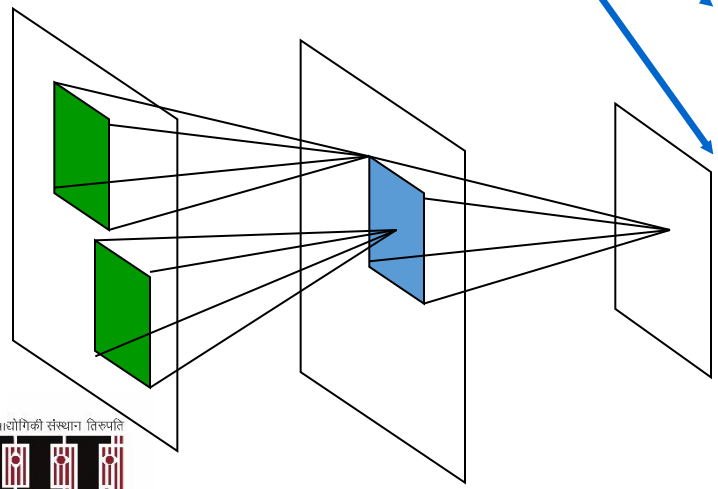
Feature extraction layer or Convolution layer

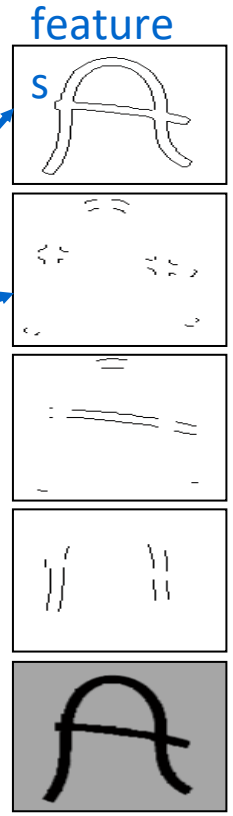
- **Detect** the **same feature** at **different positions** in the **input image**.
  - Shared weights to Reduce the number of free parameters.



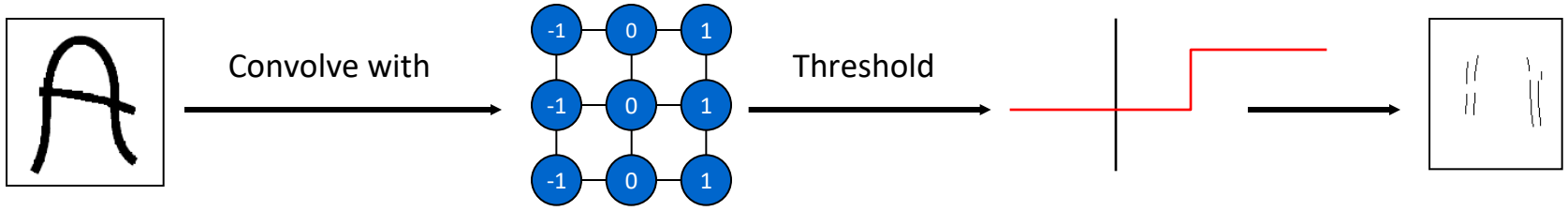


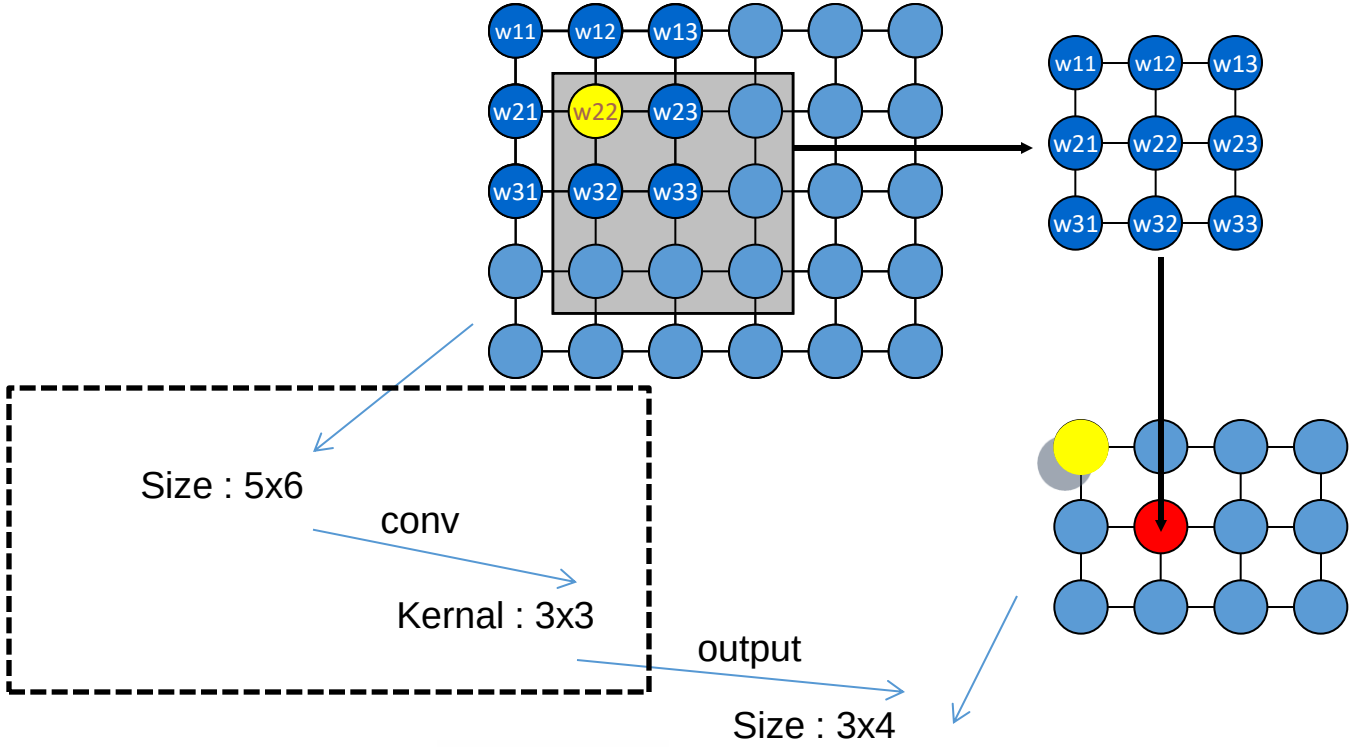




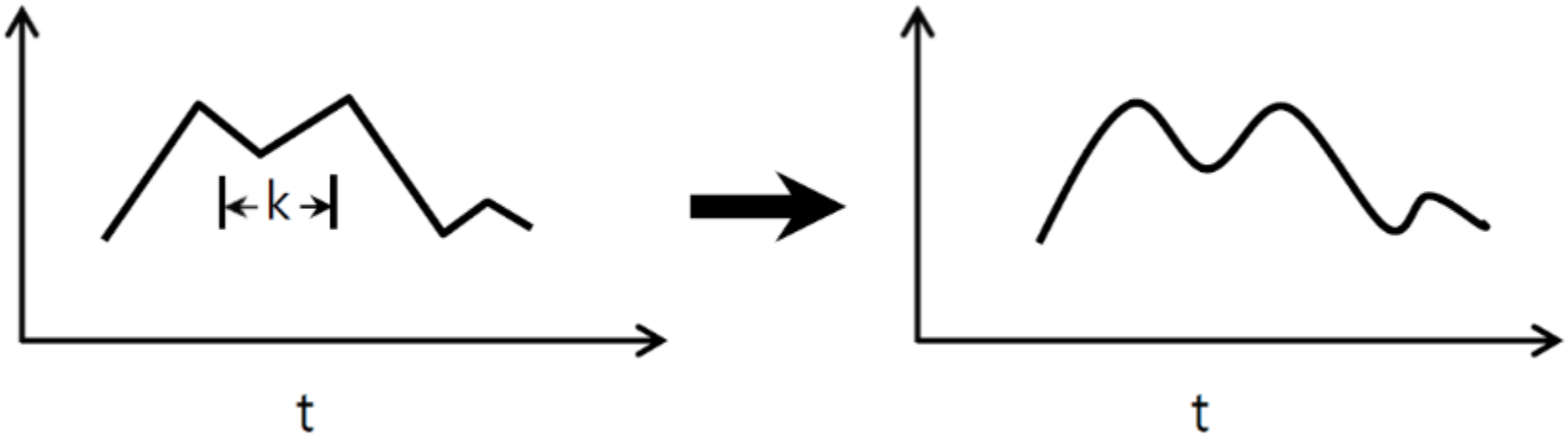


Feature Extraction

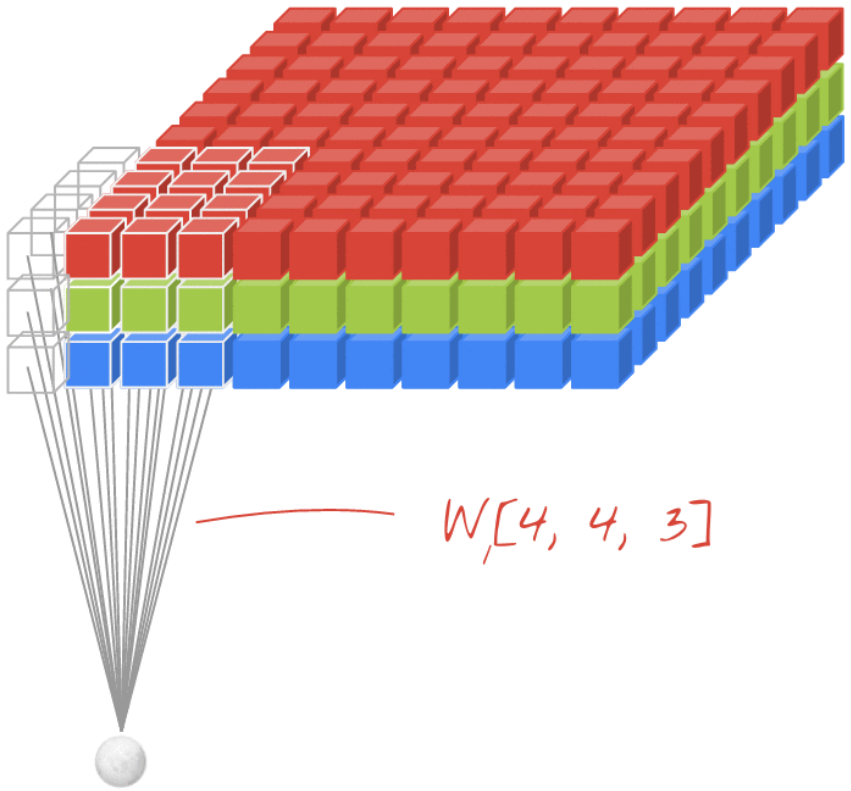




1D convolution



3D to 2D Convolution



2D Convolution

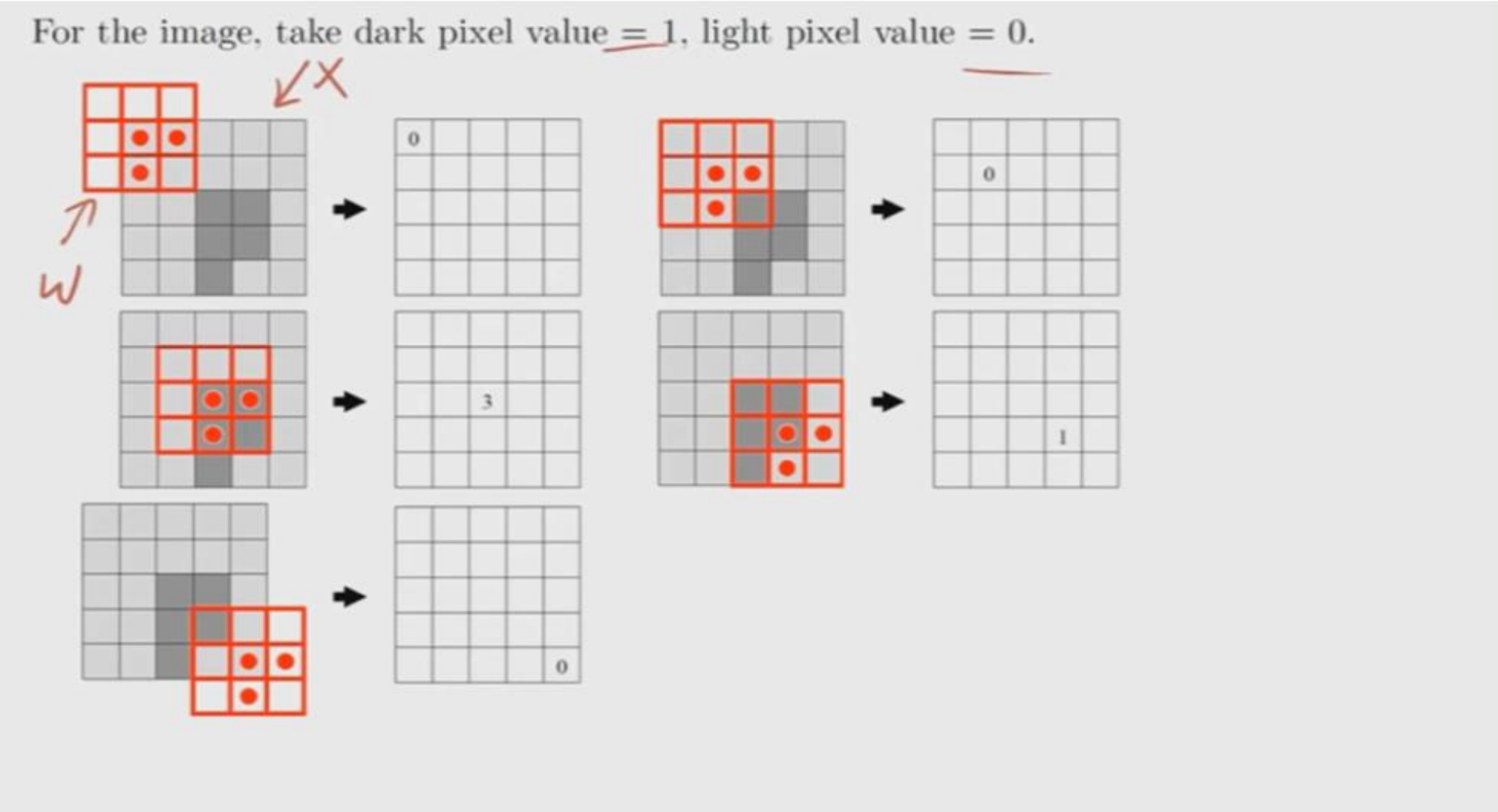
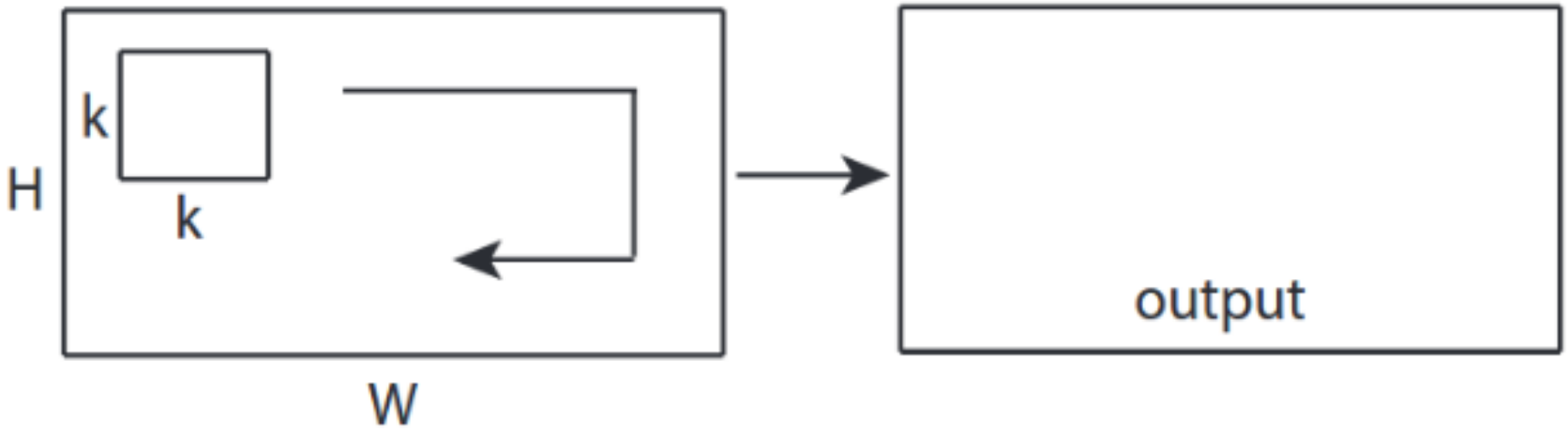
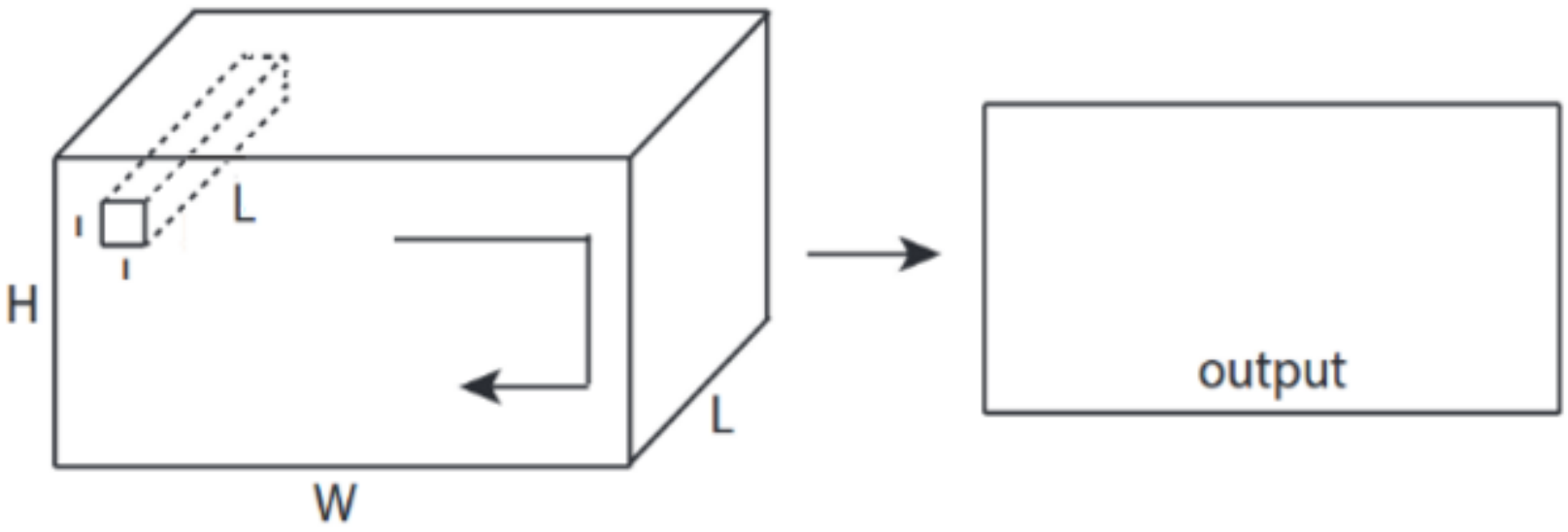


Image Credit: Prof. Nando de Forteitas

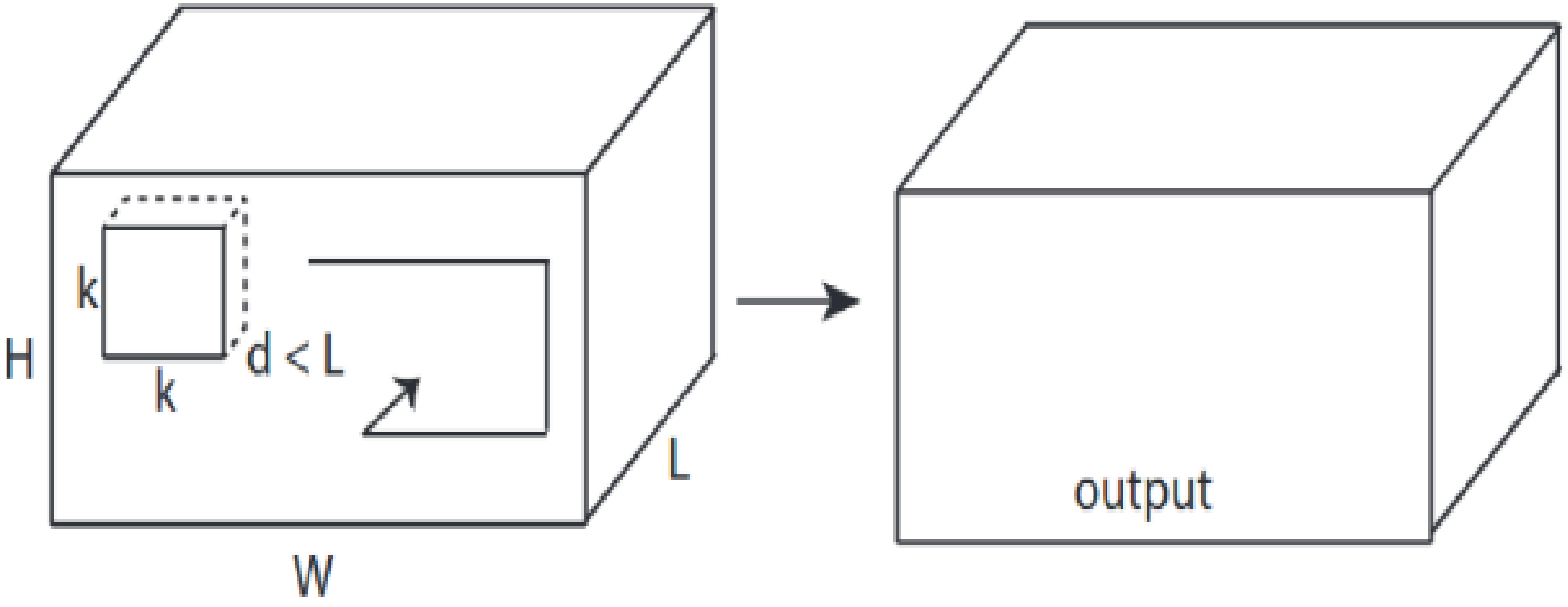
2D to 2D Convolution



3D to 2D Convolution

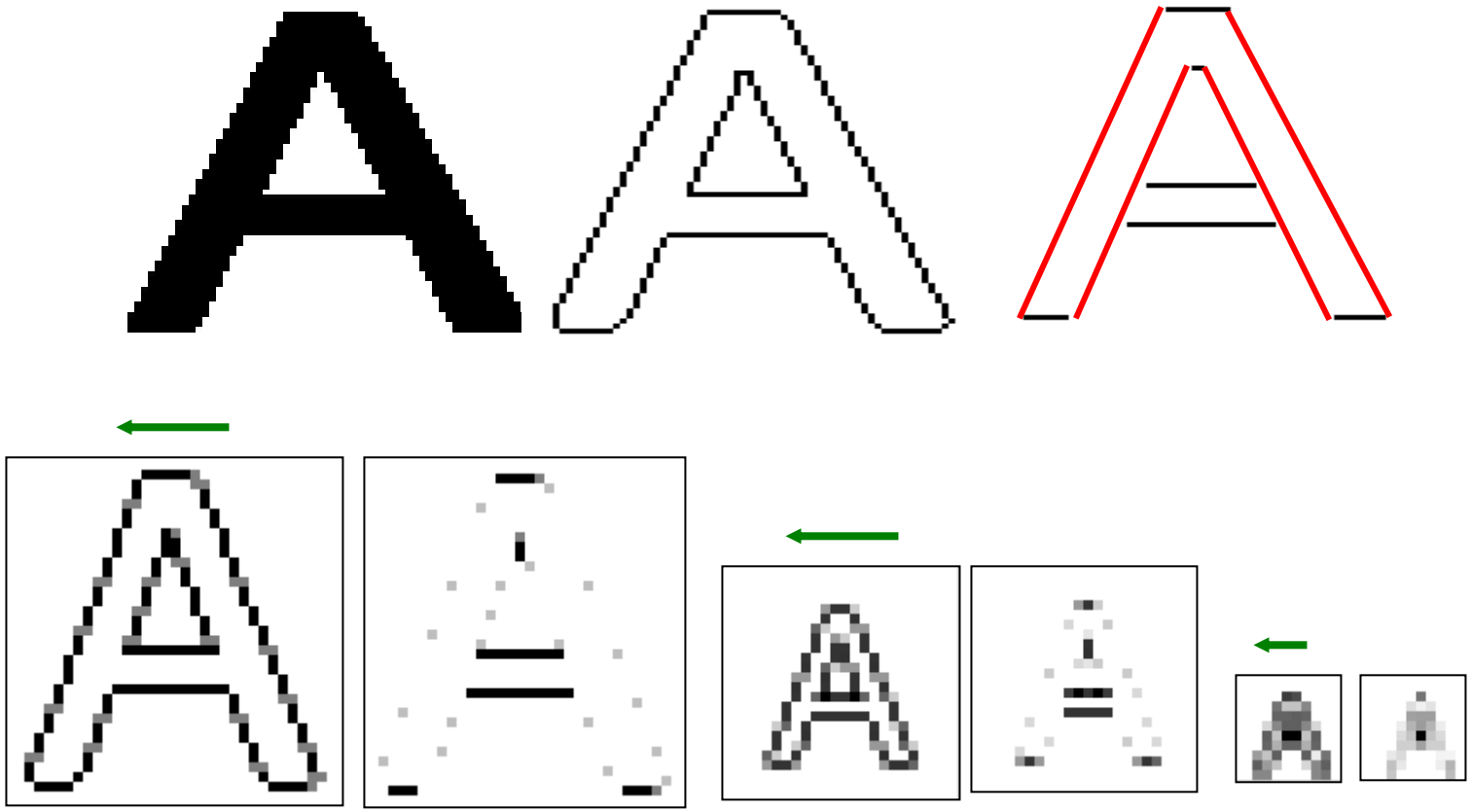


3D to 3D Convolution



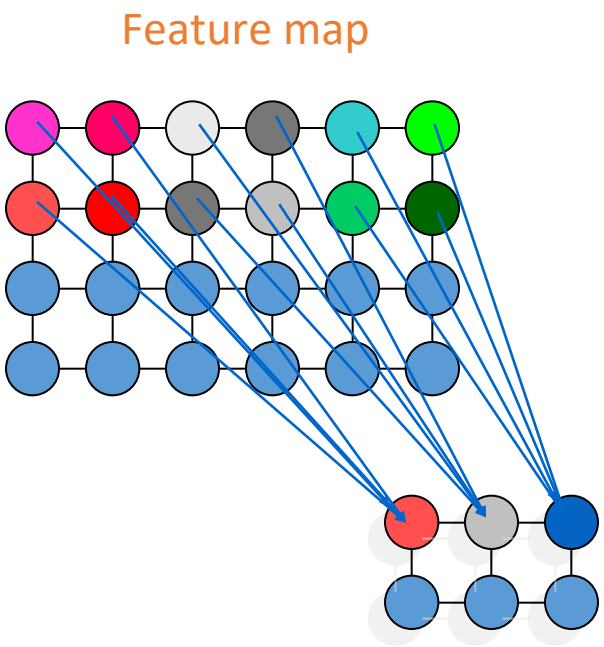
Sampling Layer

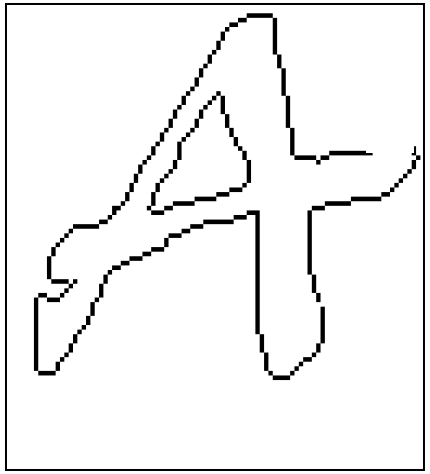
- The Subsampling layers reduce the spatial resolution of each feature map and thereby a certain degree of shift and distortion invariance is achieved.

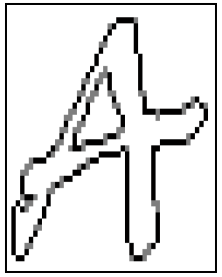


Sampling Layer

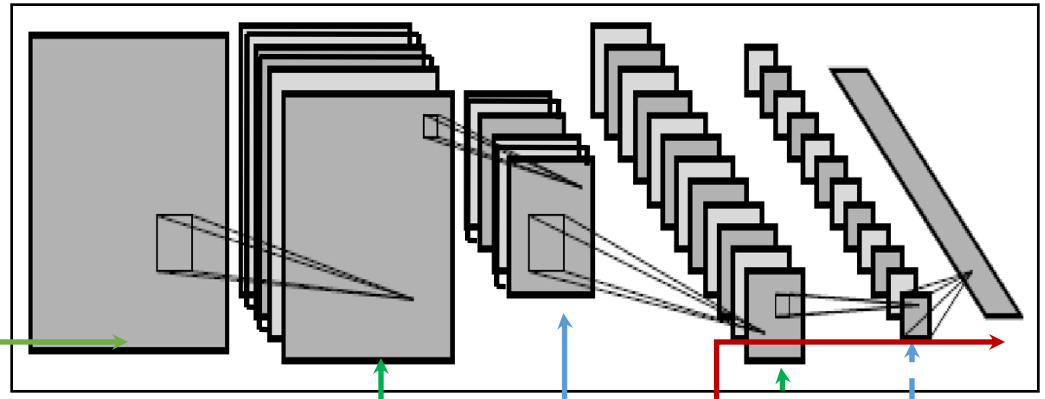
- The weight sharing is also applied in subsampling layers, reduces the effect of noises and shift or distortion







An Example : LeNet-5



- Introduced by LeCun.
- It Consists of
  - Input layer
  - Convolutional Layers (2 Nos)
  - Sampling Layers(2 Nos)
  - Fully Connected Output layer(1No)

The **MNIST** database :

(MIXED NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY)

Training Samples: 60000

Each input : 28x28 size

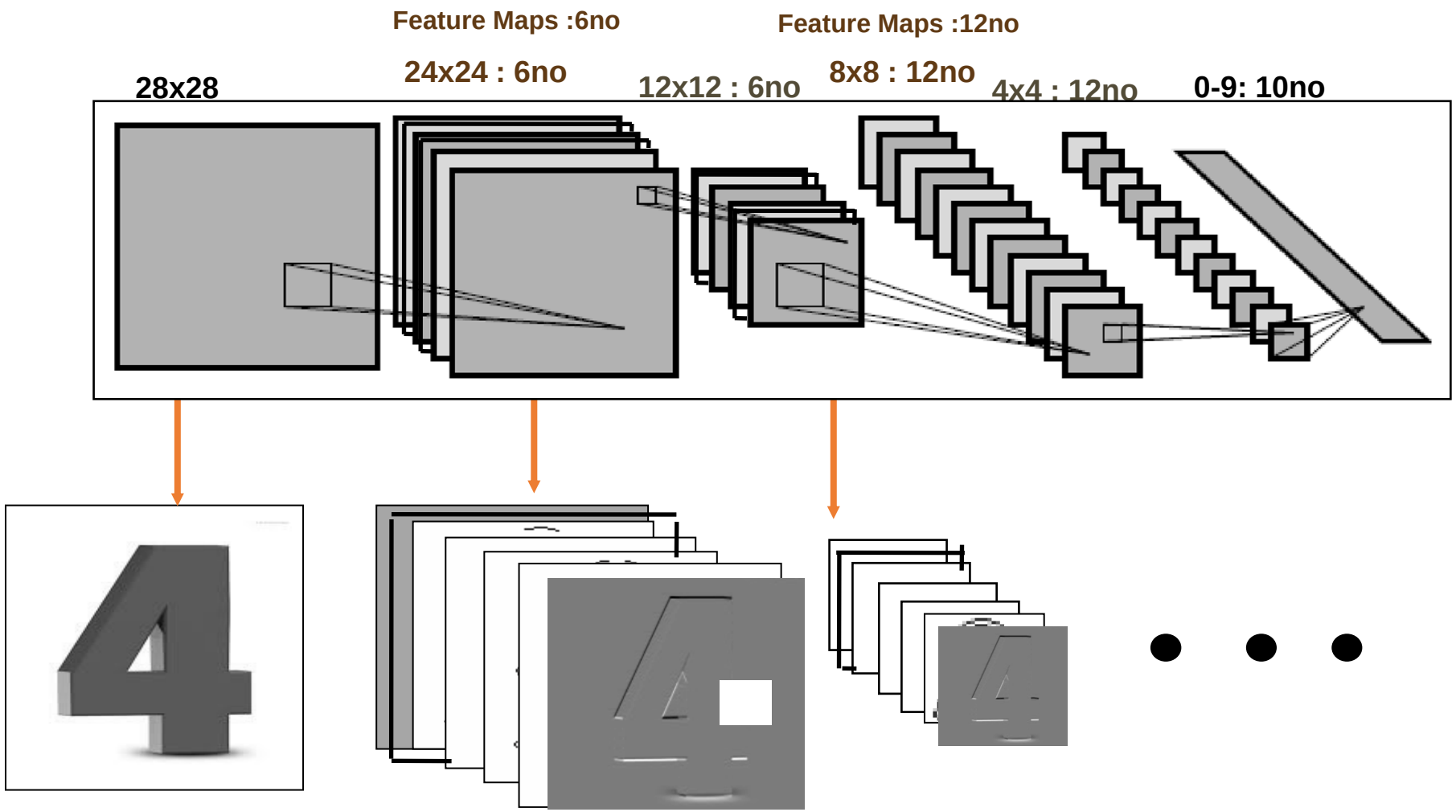
Each output: 10x1 size

Testing Samples : 10000

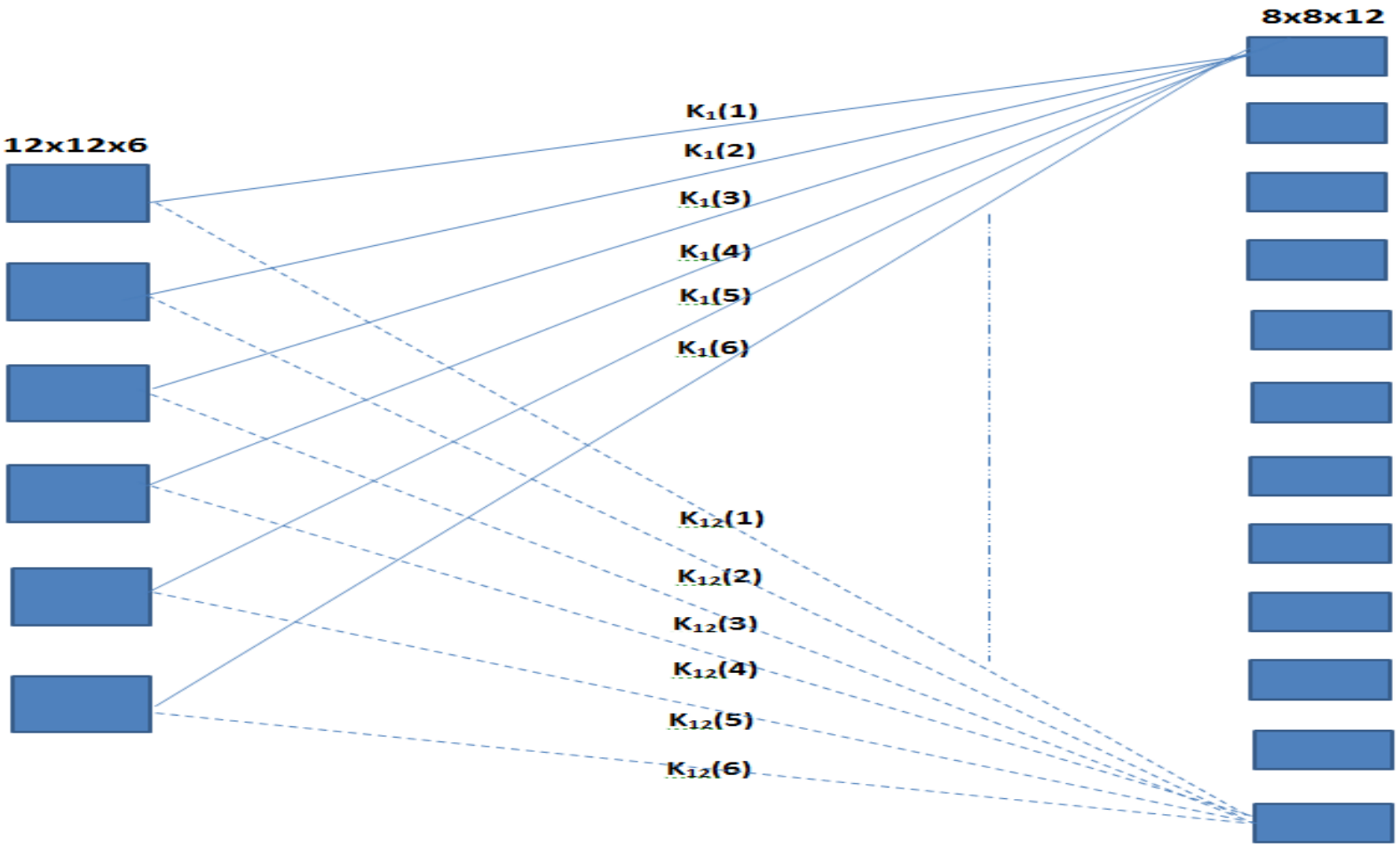
Each input : 28x28 size

Each output: 10x1 size

Note: Output is any of the digits: 0 to 9


Feature Maps :6no
Feature Maps :12no
28x28
24x24 : 6no
12x12 : 6no
8x8 : 12no
4x4 : 12no
0-9: 10no

Convolution Layer - 2



Forward Propagation

28 x 28	C_1	S_1	C_2	S_2	FC
	$[24\ x\ 24]_6$	$[12\ x\ 12]_6$	$[8\ x\ 8]_{12}$	$[4\ x\ 4]_{12}$	$10\ x\ 1$
$K\ [M\ x\ N\ x\ D]_Z$	$[5\ x\ 5\ x\ 1]_6$		$[5\ x\ 5\ x\ 6]_{12}$		

$$O[Z]_C = \boldsymbol{Sigmoid}\left(\sum_{d=1}^{D}\{I(:,\ :,\ d)\ *\ K_Z(:,\ :,\ d)\} + B(Z)\right)$$

For convolution layer – 1, (c = 1) D = 1 and Z: 1 … 6

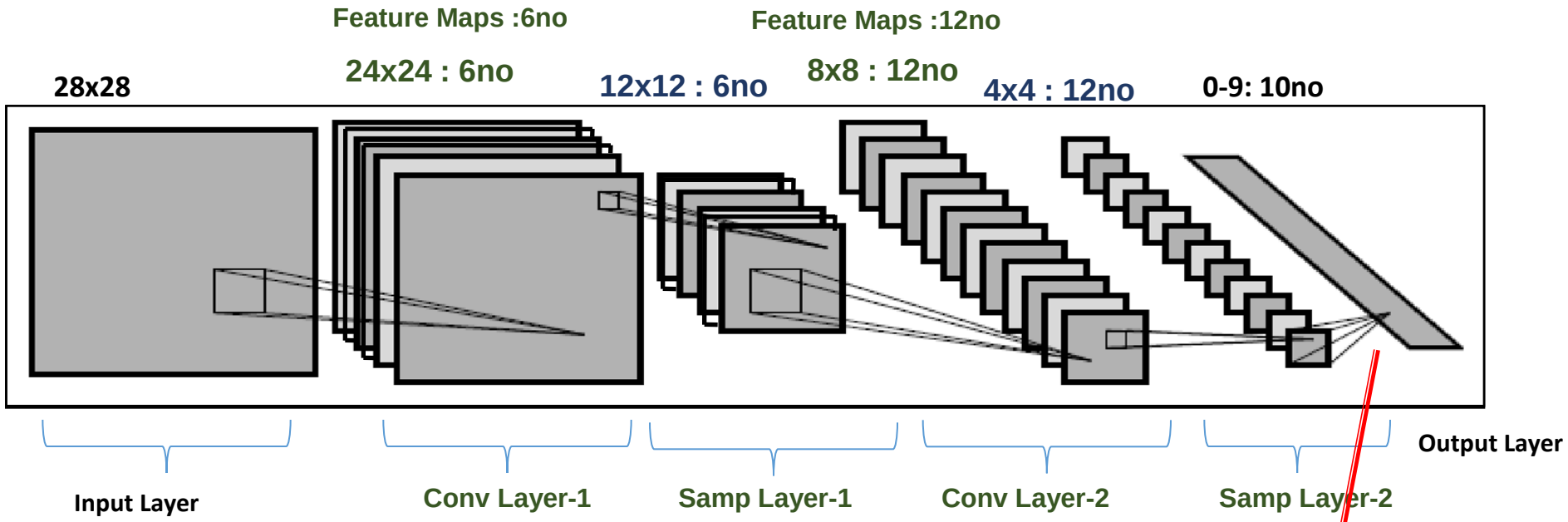
For convolution layer – 2, (c = 2) D = 6 and Z: 1 … 12

a) Feed Forward Propagation (Output calculations at each layer)

Fully Connected Layers(Final Layer)

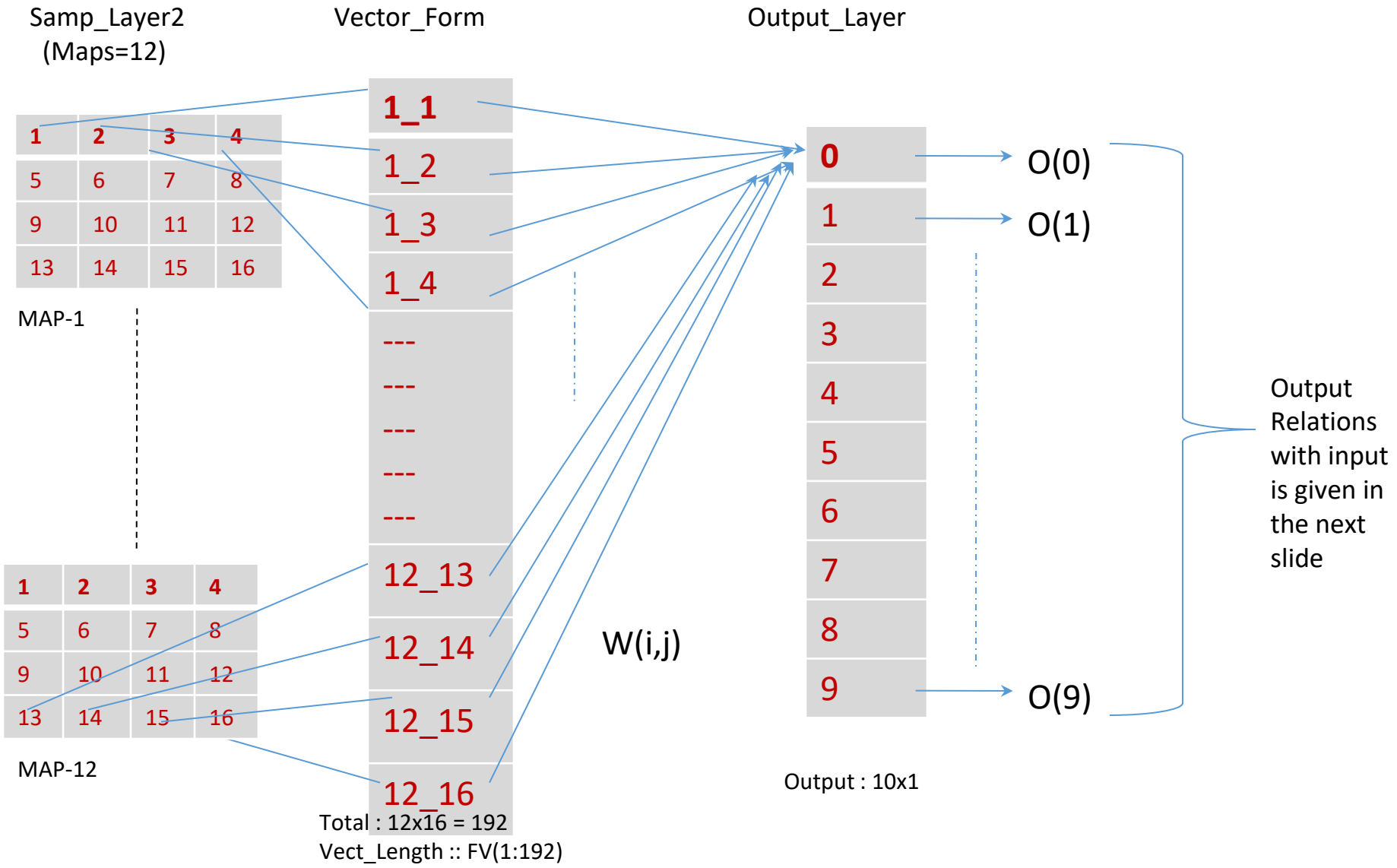
Samp_layer2 contains 12maps each 4x4 size.

Net_Function : Sigmoid_function { 1/ [1+exp(-x)] }



Fully Connected Layer (This layer is like normal neural network layer)

Fully connected(Final) Layer details: Converting to Vector Form and mapping to outputs



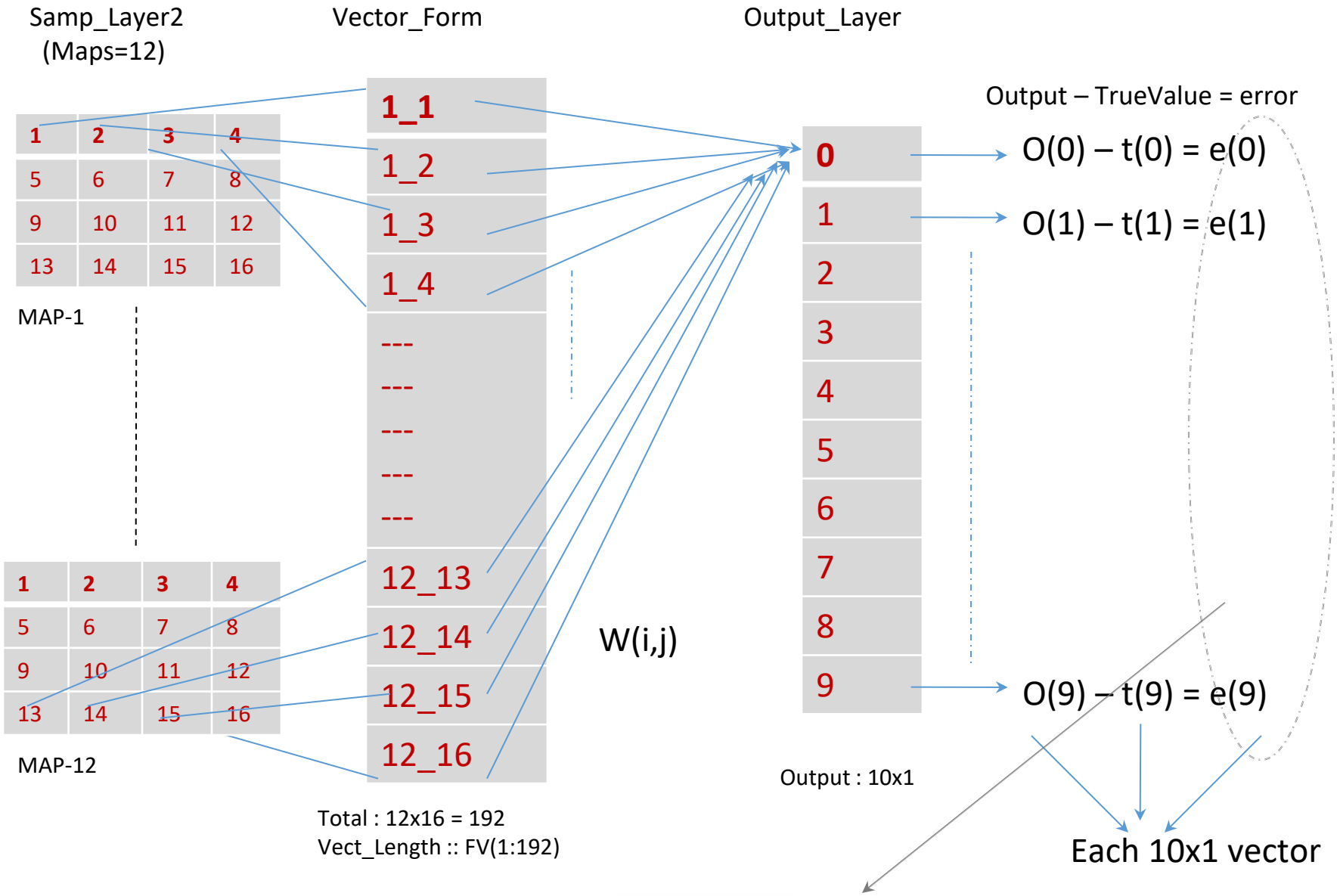
W(i,j) : Weight Connected between ith input node to jth output node.

b(j) : Bias connected to jth ouput node.

O(j) : Output at the jth node/neuron.

भारतीय प्रौद्योगिकी संस्थान तिरुपति
TIRUPATI

Training : BackPropagation : Finding delta errors at all layers)



Aim of the back propagation algorithm is that the weights, biases of all the layers are adjusted in such away that this error is as min as possible

Finding error contribution at each layer

28 x 28	C_1	S_1	C_2	S_2	FC
	$[24\ x\ 24]_6$	$[12\ x\ 12]_6$	$[8\ x\ 8]_{12}$	$[4\ x\ 4]_{12}$	$10\ x\ 1$
$K\ [M\ x\ N\ x\ D]_Z$	$[5\ x\ 5\ x\ 1]_6$		$[5\ x\ 5\ x\ 6]_{12}$		

$$\Delta FC = O_j .* (1 - O_j) .* (O_j - T_j)$$

$$\Delta S_2 = W.\Delta FC \quad [192\ x\ 10][10\ x\ 1] \rightarrow [192\ x\ 1] \rightarrow [4\ x\ 4]_{12}$$

$$\Delta C_2 = (\Delta S_2)_{US} .* O_{C2} .* (1 - O_{C2}) \quad [8\ x\ 8]_{12} .* [8\ x\ 8]_{12} .* [8\ x\ 8]_{12} \rightarrow [8\ x\ 8]_{12}$$

$$\Delta S_1(Z) = \sum_{l=1}^{L} [\Delta C_2(:,:,l) \odot K_l(:,:,Z)] \rightarrow [12\ x\ 12]_6 \quad Z = 1 \ldots 6\ \&\ L = 12$$

$$\Delta C_1 = (\Delta S_1)_{US} .* O_{C1} .* (1 - O_{C1}) \quad [24\ x\ 24]_6 .* [24\ x\ 24]_6 .* [24\ x\ 24]_6 \rightarrow [24\ x\ 24]_6$$

Gradient and update rule

$$(\Delta K)_C = \Delta C * I \qquad C = 1 \rightarrow [24\ x\ 24]_6 * [28\ x\ 28] \rightarrow [5\ x\ 5]_6$$

$$C = 2 \rightarrow [8\ x\ 8]_{12} * [12\ x\ 12]_6 \rightarrow [5\ x\ 5\ x\ 6]_{12}$$

$$[8\ x\ 8, 1] * [12\ x\ 12, 1] \rightarrow [5\ x\ 5, 1]_1 \cdots\cdots [8\ x\ 8, 12] * [12\ x\ 12, 6] \rightarrow [5\ x\ 5, 6]_{12}$$

$$(\Delta B)_C = Avg\ (\Delta K)_C \qquad C = 1 \rightarrow [6\ x\ 1] \qquad C = 2 \rightarrow [12\ x\ 1]$$

$$(\Delta W)_{FC} = \Delta FC\ .I_{FC} \qquad \rightarrow [10\ x\ 1].[1\ x\ 192] \rightarrow [10\ x\ 192]$$

$$(\Delta B)_{FC} = Avg\ (\Delta W)_{FC}$$

$$W^{new} = W^{old} - \alpha\, {}^{\delta W}/_{\delta\theta}$$

भारतीय प्रौद्योगिकी संस्थान तिरुपति

TIRUPATI

Convolution and Correlation

X =

x_1	x_2	x_3	x_4
x_5	x_6	x_7	x_8
x_9	x_{10}	x_{11}	x_{12}
x_{13}	x_{14}	x_{15}	x_{16}

W =

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

X * W =

Y_1	Y_2
Y_3	Y_4

ΔY =

		ΔY_1	ΔY_2		
		ΔY_3	ΔY_4		

W =

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

ΔY ⊙ W =

Δx_1	Δx_2	Δx_3	Δx_4
Δx_5	Δx_6	Δx_7	Δx_8
Δx_9	Δx_{10}	Δx_{11}	Δx_{12}
Δx_{13}	Δx_{14}	Δx_{15}	Δx_{16}

$$
\begin{aligned}
Y_1 &= w_9x_1 + w_8x_2 + w_7x_3 + w_6x_5 + w_5x_6 + w_4x_7 + w_3x_9 + w_2x_{10} + w_1x_{11} \\
Y_2 &= w_9x_2 + w_8x_3 + w_7x_4 + w_6x_6 + w_5x_7 + w_4x_8 + w_3x_{10} + w_2x_{11} + w_1x_{12} \\
Y_3 &= w_9x_5 + w_8x_6 + w_7x_7 + w_6x_9 + w_5x_{10} + w_4x_{11} + w_3x_{13} + w_2x_{14} + w_1x_{15} \\
Y_4 &= w_9x_6 + w_8x_7 + w_7x_8 + w_6x_{10} + w_5x_{11} + w_4x_{12} + w_3x_{14} + w_2x_{15} + w_1x_{16}
\end{aligned}
$$

$$
\begin{aligned}
\Delta x_1 &= w_9\Delta Y_1 & \Delta x_2 &= w_9\Delta Y_2 + w_8\Delta Y_1 & \Delta x_6 &= w_9\Delta Y_4 + w_8\Delta Y_3 + w_6\Delta Y_2 + w_5\Delta Y_1 & \Delta x_9 &= w_6\Delta Y_3 + w_3\Delta Y_1 \\
\Delta x_4 &= w_7\Delta Y_2 & \Delta x_3 &= w_8\Delta Y_2 + w_7\Delta Y_1 & \Delta x_7 &= w_8\Delta Y_4 + w_7\Delta Y_3 + w_5\Delta Y_2 + w_4\Delta Y_1 & \Delta x_{12} &= w_4\Delta Y_4 + w_1\Delta Y_2 \\
\Delta x_{13} &= w_3\Delta Y_3 & \Delta x_5 &= w_9\Delta Y_3 + w_6\Delta Y_1 & \Delta x_{10} &= w_6\Delta Y_4 + w_3\Delta Y_2 + w_5\Delta Y_3 + w_2\Delta Y_1 & \Delta x_{14} &= w_3\Delta Y_4 + w_2\Delta Y_3 \\
\Delta x_{16} &= w_1\Delta Y_4 & \Delta x_8 &= w_7\Delta Y_4 + w_4\Delta Y_2 & \Delta x_{11} &= w_5\Delta Y_4 + w_2\Delta Y_2 + w_4\Delta Y_3 + w_1\Delta Y_1 & \Delta x_{15} &= w_2\Delta Y_4 + w_1\Delta Y_3
\end{aligned}
$$

Kernel Weights update in CNN

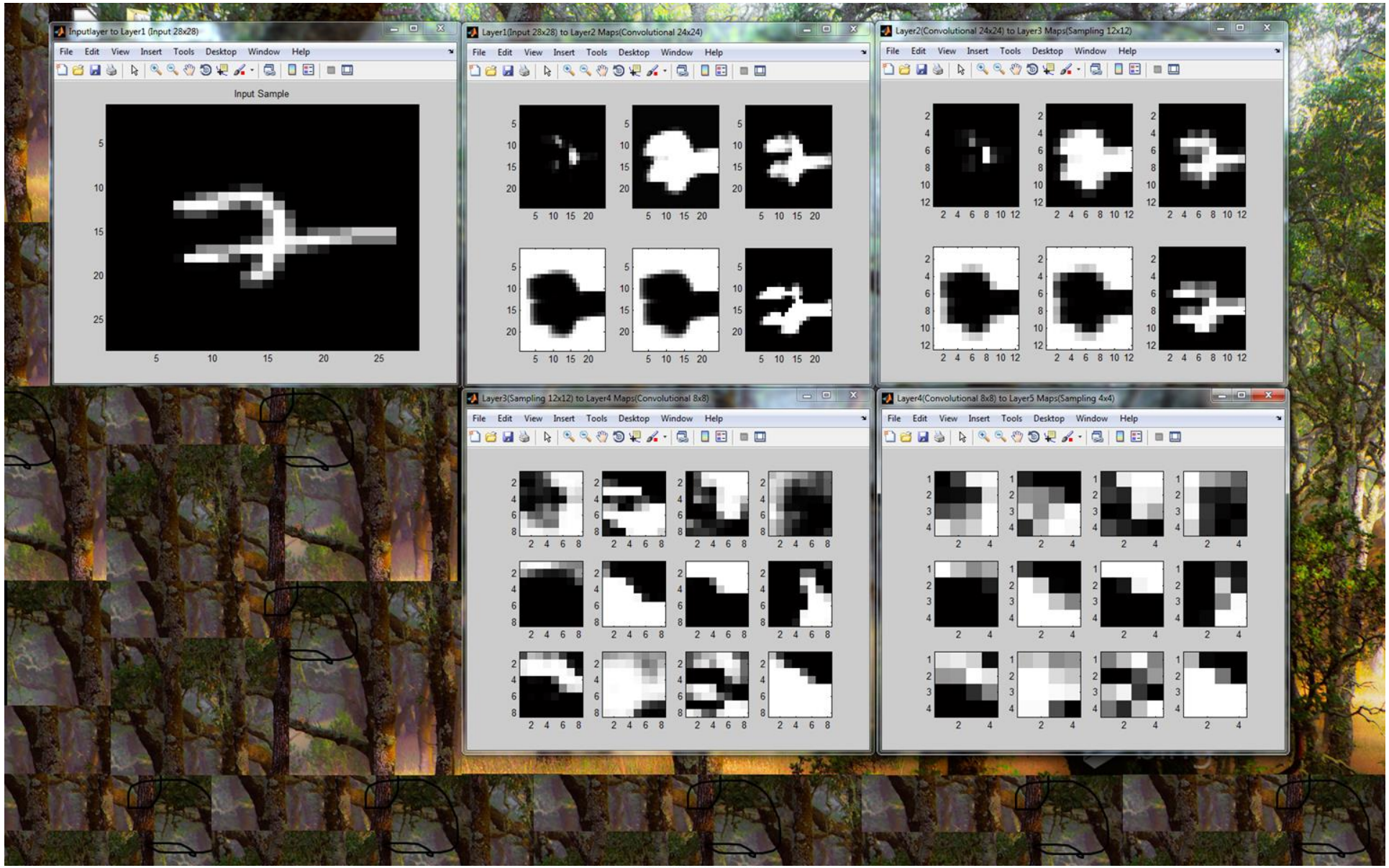
$$\begin{bmatrix} X_1 & X_2 & X_3 & X_4 & X_5 \\ X_6 & X_7 & X_8 & X_9 & X_{10} \\ X_{11} & X_{12} & X_{13} & X_{14} & X_{15} \\ X_{16} & X_{17} & X_{18} & X_{19} & X_{20} \\ X_{21} & X_{22} & X_{23} & X_{24} & X_{25} \end{bmatrix} * \begin{bmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{bmatrix} = \begin{bmatrix} y_1 & y_2 & y_3 \\ y_4 & y_5 & y_6 \\ y_7 & y_8 & y_9 \end{bmatrix}$$

$$\begin{aligned}
y_1 &= w_9X_1 + w_8X_2 + w_7X_3 + w_6X_6 + w_5X_7 + w_4X_8 + w_3X_{11} + w_2X_{12} + w_1X_{13} \\
y_2 &= w_9X_2 + w_8X_3 + w_7X_4 + w_6X_7 + w_5X_8 + w_4X_9 + w_3X_{12} + w_2X_{13} + w_1X_{14} \\
y_3 &= w_9X_3 + w_8X_4 + w_7X_5 + w_6X_8 + w_5X_9 + w_4X_{10} + w_3X_{13} + w_2X_{14} + w_1X_{15} \\
y_4 &= w_9X_6 + w_8X_7 + w_7X_8 + w_6X_{11} + w_5X_{12} + w_4X_{13} + w_3X_{16} + w_2X_{17} + w_1X_{18} \\
y_5 &= w_9X_7 + w_8X_8 + w_7X_9 + w_6X_{12} + w_5X_{13} + w_4X_{14} + w_3X_{17} + w_2X_{18} + w_1X_{19} \\
y_6 &= w_9X_8 + w_8X_9 + w_7X_{10} + w_6X_{13} + w_5X_{14} + w_4X_{15} + w_3X_{18} + w_2X_{19} + w_1X_{20} \\
y_7 &= w_9X_{11} + w_8X_{12} + w_7X_{13} + w_6X_{16} + w_5X_{17} + w_4X_{18} + w_3X_{21} + w_2X_{22} + w_1X_{23} \\
y_8 &= w_9X_{12} + w_8X_{13} + w_7X_{14} + w_6X_{17} + w_5X_{18} + w_4X_{19} + w_3X_{22} + w_2X_{23} + w_1X_{24} \\
y_9 &= w_9X_{13} + w_8X_{14} + w_7X_{15} + w_6X_{18} + w_5X_{19} + w_4X_{20} + w_3X_{23} + w_2X_{24} + w_1X_{25}
\end{aligned}$$

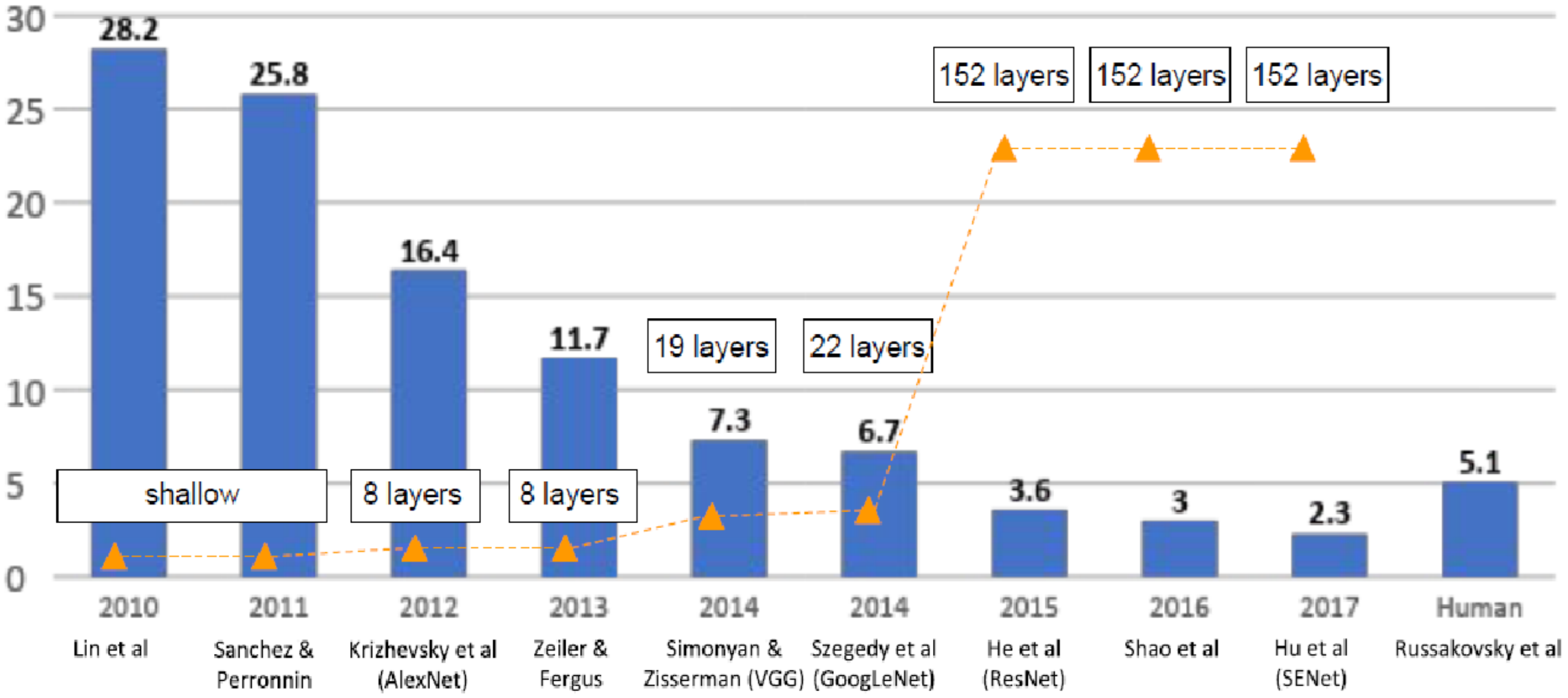
$$\begin{aligned}
\Delta w_1 &= \Delta y_9X_{25} + \Delta y_8X_{24} + \Delta y_7X_{23} + \Delta y_6X_{20} + \Delta y_5X_{19} + \Delta y_4X_{18} + \Delta y_3X_{15} + \Delta y_{14}X_{14} + \Delta y_1X_{13} \\
\Delta w_2 &= \Delta y_9X_{24} + \Delta y_8X_{23} + \Delta y_7X_{22} + \Delta y_6X_{19} + \Delta y_5X_{18} + \Delta y_4X_{17} + \Delta y_3X_{14} + \Delta y_2X_{13} + \Delta y_1X_{12} \\
\Delta w_3 &= \Delta y_9X_{23} + \Delta y_8X_{22} + \Delta y_7X_{21} + \Delta y_6X_{18} + \Delta y_5X_{17} + \Delta y_4X_{16} + \Delta y_3X_{13} + \Delta y_2X_{12} + \Delta y_1X_{11} \\
\Delta w_4 &= \Delta y_9X_{20} + \Delta y_8X_{19} + \Delta y_7X_{18} + \Delta y_6X_{15} + \Delta y_5X_{14} + \Delta y_4X_{13} + \Delta y_3X_{10} + \Delta y_2X_9 + \Delta y_1X_8 \\
\Delta w_5 &= \Delta y_9X_{19} + \Delta y_8X_{18} + \Delta y_7X_{17} + \Delta y_6X_{14} + \Delta y_5X_{13} + \Delta y_4X_{12} + \Delta y_3X_9 + \Delta y_2X_8 + \Delta y_1X_7 \\
\Delta w_6 &= \Delta y_9X_{18} + \Delta y_8X_{17} + \Delta y_7X_{16} + \Delta y_6X_{13} + \Delta y_5X_{12} + \Delta y_4X_{11} + \Delta y_3X_8 + \Delta y_2X_7 + \Delta y_1X_6 \\
\Delta w_7 &= \Delta y_9X_{15} + \Delta y_8X_{14} + \Delta y_7X_{13} + \Delta y_6X_{10} + \Delta y_5X_9 + \Delta y_4X_8 + \Delta y_3X_5 + \Delta y_2X_4 + \Delta y_1X_3 \\
\Delta w_8 &= \Delta y_9X_{14} + \Delta y_8X_{13} + \Delta y_7X_{12} + \Delta y_6X_9 + \Delta y_5X_8 + \Delta y_4X_7 + \Delta y_3X_4 + \Delta y_2X_3 + \Delta y_1X_2 \\
\Delta w_9 &= \Delta y_9X_{13} + \Delta y_8X_{12} + \Delta y_7X_{11} + \Delta y_6X_8 + \Delta y_5X_7 + \Delta y_4X_6 + \Delta y_3X_3 + \Delta y_2X_2 + \Delta y_1X_1
\end{aligned}$$

$$\begin{bmatrix} X_{25} & X_{24} & X_{23} & X_{22} & X_{21} \\ X_{20} & X_{19} & X_{18} & X_{17} & X_{16} \\ X_{15} & X_{14} & X_{13} & X_{12} & X_{11} \\ X_{10} & X_9 & X_8 & X_7 & X_6 \\ X_5 & X_4 & X_3 & X_2 & X_1 \end{bmatrix} * \begin{bmatrix} \Delta y_1 & \Delta y_2 & \Delta y_3 \\ \Delta y_4 & \Delta y_5 & \Delta y_6 \\ y\Delta_7 & \Delta y_8 & \Delta y_9 \end{bmatrix} = \begin{bmatrix} \Delta w_1 & \Delta w_2 & \Delta w_3 \\ \Delta w_4 & \Delta w_5 & \Delta w_6 \\ \Delta w_7 & \Delta w_8 & \Delta w_9 \end{bmatrix}$$

CNN: All Layer Maps For a sample



ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



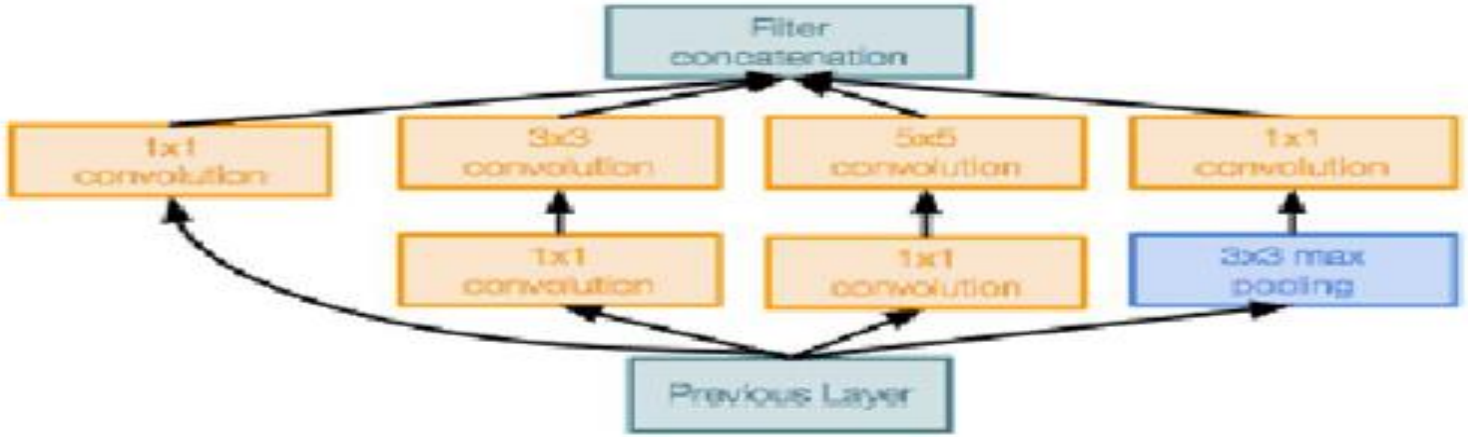
Slide Credit: Fei Fei Li

Popular Datasets (http://cs231n.stanford.edu/project.html)

- Meta Pointer: A large collection organized by CV Datasets.
- Yet another Meta pointer
- ImageNet: a large-scale image dataset for visual recognition organized by WordNet hierarchy
- SUN Database: a benchmark for scene recognition and object detection with annotated scene categories and segmented objects
- Places Database: a scene-centric database with 205 scene categories and 2.5 millions of labelled images
- NYU Depth Dataset v2: a RGB-D dataset of segmented indoor scenes
- Microsoft COCO: a new benchmark for image recognition, segmentation and captioning
- Flickr100M: 100 million creative commons Flickr images
- Labeled Faces in the Wild: a dataset of 13,000 labeled face photographs
- Human Pose Dataset: a benchmark for articulated human pose estimation
- YouTube Faces DB: a face video dataset for unconstrained face recognition in videos
- UCF101: an action recognition data set of realistic action videos with 101 action categories
- HMDB-51: a large human motion dataset of 51 action classes
- ActivityNet: A large-scale video dataset for human activity understanding
- Moments in Time: A dataset of one million 3-second videos

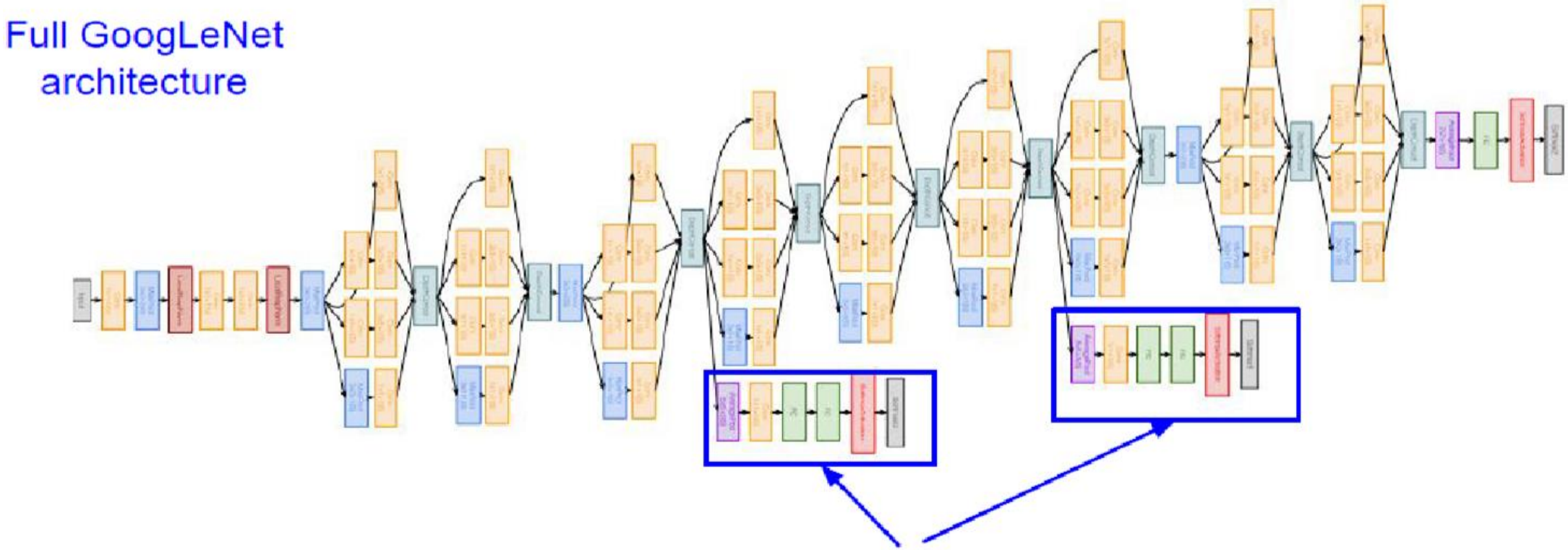
Slide Credit: Fei Fei Li

Inception Module



Inception module

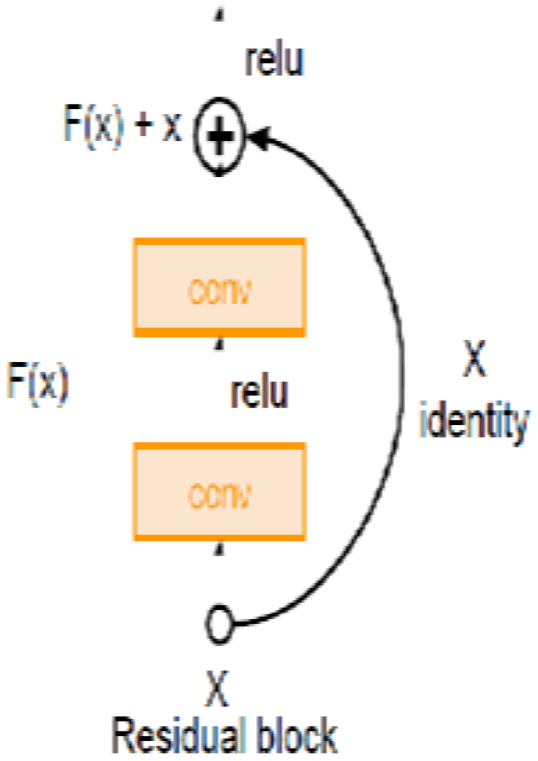
GoogLe Net

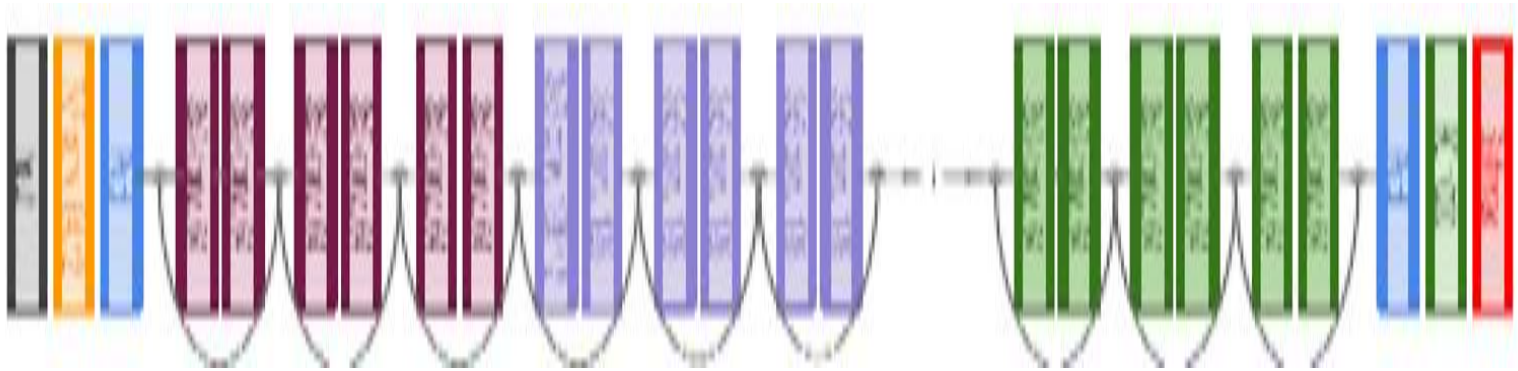


Auxiliary classification outputs to inject additional gradient at lower layers
(AvgPool-1x1Conv-FC-FC-Softmax)

Slide Credit: Fei Fei Li

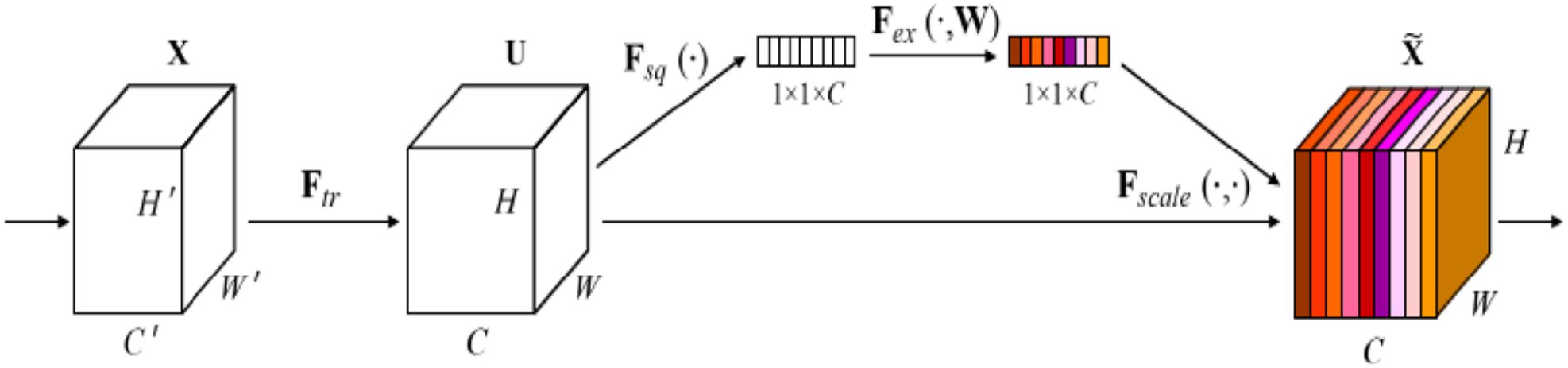
Skip Connection and ResNet





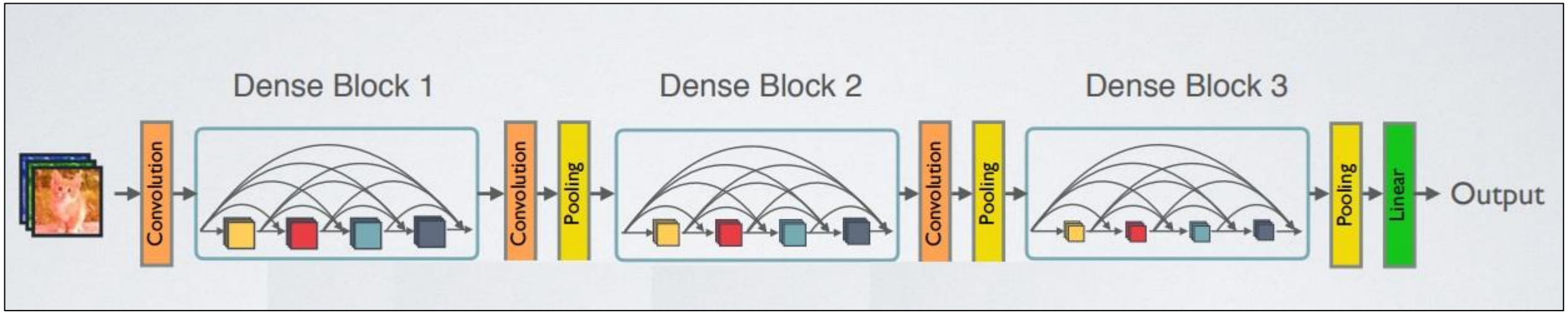
Squeeze and Excitation Block :

https://arxiv.org/abs/1709.01507

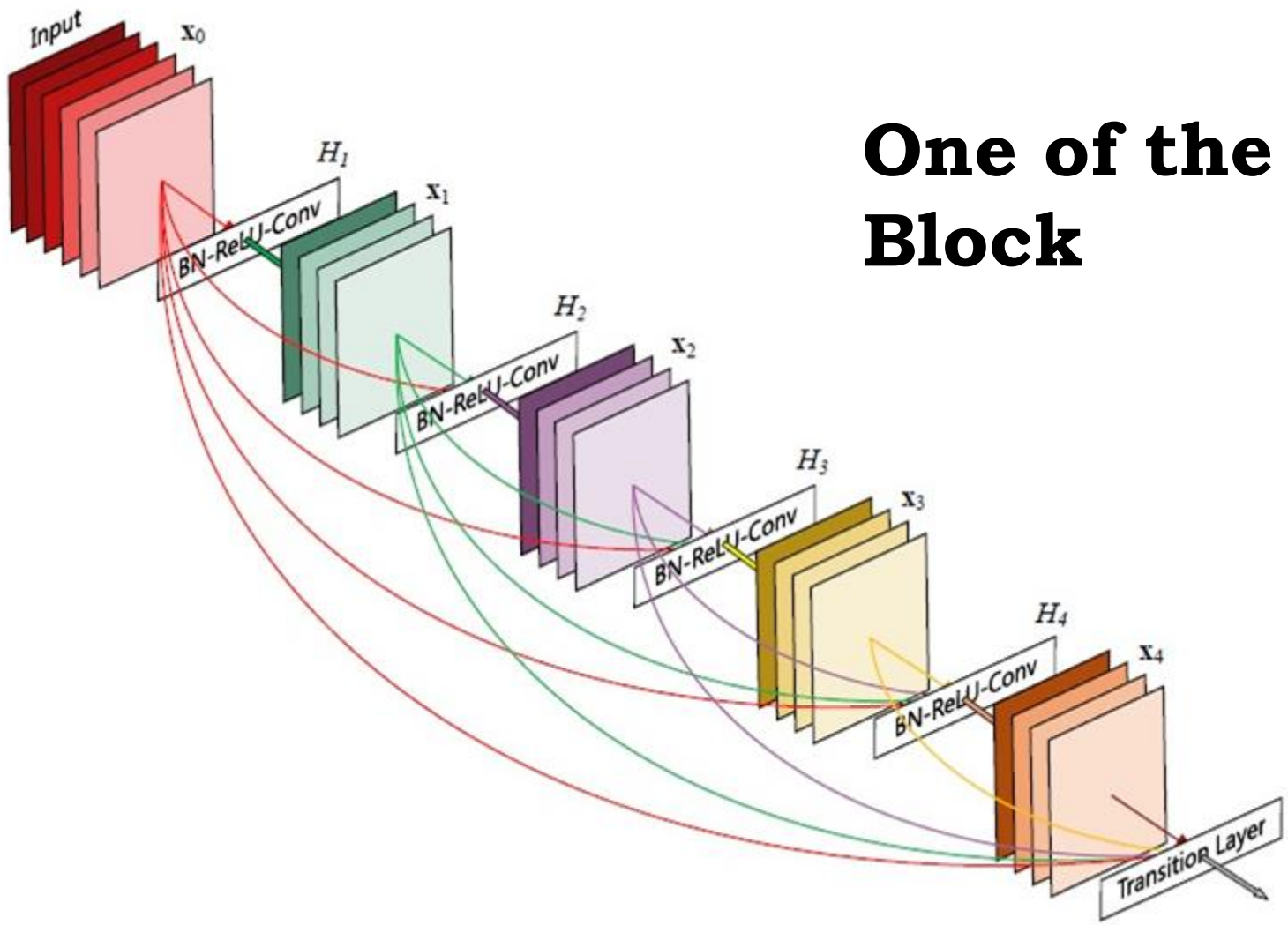


DenseNet

https://arxiv.org/pdf/1608.06993.pdf



One of the Dense Block



Ref : https://arxiv.org/pdf/1608.06993.pdf

References

Text Book:

Deep Learning, Ian Goodfellow and Yoshua Bengio and Aaron Courville, MIT Press, 2016.

http://www.deeplearningbook.org/ Can be ordered from Amazon.

References:

1. **Prof. Andrew NG Video Lectures on Deep Learning and Sequence Models**
2. **Prof. Fei-Fei Stanford group video lectures on Convolution NNs for Visual Recognition** https://www.youtube.com/watch?v=vT1JzLTH4G4
3. **Slides from Prof. Fei-Fei group** http://cs231n.stanford.edu/syllabus.html
4. **Slides on Deep learning book** http://www.deeplearningbook.org/lecture_slides.html
5. **Video discussions on Deep Learning book** https://www.youtube.com/channel/UCF9O8Vj-FEbRDA5DcDGz-Pg/videos
6. **Geoffrey Hinton, Yoshua Bengio, and Yan LeCun Deep learning collections** http://deeplearning.net/

Reference Papers

- ImageNet Classification with Deep Convolutional Neural Networks, (Alex Net), Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton, NIPS 2012.
- Going Deeper with Convolutions, (Google Net), Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, Andrew Rabinovich, 19-Sept-2014.
- Very Deep Convolutional Networks for Large-Scale Image Recognition (VGG Net), Karen Simonyan, Andrew Zisserman, ICLR 2015.
- Deep Residual Learning for Image Recognition, Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, CVPR 2015.
- Densely Connected Convolutional Networks, Gao Huang, Zhuang Liu, Laurens van der Maaten, Kilian Q. Weinberger, CVPR 2017.
- Squeeze-and-Excitation Networks, Jie Hu, Li Shen, Gang Sun, 2018.
- Fully Convolutional Networks for Semantic Segmentation, Jonathan Long, Evan Shelhamer, Trevor Darrell, CVPR 2015.
- DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs, IEEE Transactions on Pattern Analysis and Machine Intelligence , Volume: 40, Issue: 4, April 1 2018 .
- You Only Look Once: Unified, Real-Time Object Detection, *Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi*; The IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.
- Depth Map Prediction from a Single Image using a Multi-Scale Deep Network, David Eigen, NIPS 2014.

Popular References (http://cs231n.stanford.edu/project.html)

- **Image Classification**: [Krizhevsky et al.], [Russakovsky et al.], [Szegedy et al.], [Simonyan et al.], [He et al.], [Huang et al.], [Hu et al.] [Zoph et al.]
- **Object detection**: [Girshick et al.], [Ren et al.], [He et al.]
- **Image segmentation**: [Long et al.] [Noh et al.] [Chen et al.]
- **Scene classification**: [Zhou et al.]
- **Face recognition**: [Taigman et al.] [Schroff et al.] [Parkhi et al.]
- **Depth estimation**: [Eigen et al.]
- **Image-to-sentence generation**: [Karpathy and Fei-Fei], [Donahue et al.], [Vinyals et al.] [Xu et al.] [Johnson et al.]
- **Visualization and optimization**: [Szegedy et al.], [Nguyen et al.], [Zeiler and Fergus], [Goodfellow et al.], [Schaul et al.]
- **Video classification**: [Karpathy et al.], [Simonyan and Zisserman] [Tran et al.] [Carreira et al.] [Wang et al.]

Information for mini-Projects

- Tutorials, Reviews and Ted talks can be seen here: http://deeplearning.net/reading-list/tutorials/
- An extensive reading list till 2015 can be seen here: http://deeplearning.net/reading-list/
- CV Data Sets can be found from : http://deeplearning.net/datasets/
- Excellent Pointers for Projects http://cs231n.stanford.edu/project.html

Papers with Codes

- Papers with Code @ https://paperswithcode.com/sota
- Image Denoising: https://paperswithcode.com/task/image-denoising
- Object Detection: https://paperswithcode.com/task/object-detection
- Segmentation: https://paperswithcode.com/task/semantic-segmentation
- Visual Object Tracking: https://paperswithcode.com/task/visual-object-tracking

Questions?